Administration d'Oracle
1

Pr Mohamed CHAKRAOUI

Plan

- Installation du SGBD Oracle;
- Découvrir l'architecture du serveur Oracle ;
- Découvrir l'outil d'administration Oracle SQL*Plus;
- Gérer une instance Oracle;
- Gérer les fichiers d'une base de données Oracle;
- Gérer les unités de stockage logique;
- Voir la différence entre les versions dernières versions d'Oracle;
- Création d'une base de données .
- …

Webographie

https://fr.wikipedia.org/wiki/Oracle_(entreprise)

https://www.editions-eni.fr/open/mediabook.aspx?idR=7b598a5dd2dc3c9b11f4897259948d39

https://docs.oracle.com/database/121/ADMQS/GUID-DBC8C390-D3B7-40A6-B625-26AE44AA45E1.htm#ADMQS021

https://www.oracle.com/virtualization/technologies/virtual-appliances.html

Préparation de l'environnement de travail

Installation d'racle 19c et 12g voir TP1

Préparation de l'environnement de travail

- Système d'exploitation
  - Unix, linux, Windows…
  - Nombre de bit 32, 64
- Disque dur au moins 5 Go d'espace libre (10Go pour la machine virtuelle + SGBD)
- RAM : 30% libre
- Espace SWAP: c'est une partie de la mémoire qui existe dans le disque dur

Prérequis / Linux

- User avec groupe DBA
- \TMP espace temporaire(500 mb)
- Package Kernel
- Sécurité Kernel (20% de CPU)
- Protocol X (exemple : Xmanager)
- Pour Windows il faut un user admin et non pas un user a un rôle admin
- Il faut choisir le binaires d'Oracle selon le OS + le nombre de bit

Variables d'environnements

- Comme tous les OS, Windows possède des **variables d'environnement** qui sont accessibles pour tout le système. Par exemple: USERNAME contient le nom du user connecté, COMPUTERNAME contient le nom de l'ordi, OS le nom de l'OS.

- Lister les variables d'environnement:

Dans une console Windows (Menu démarrer > Exécuter > cmd), il suffit de taper set pour voir la liste des variables d'environnement actuelle.

Variables d'environnements

- **Voir la valeur d'une variable**:
  - set NOM_DE_LA_VARIABLE
  - Donc l'exemple ci-dessous affichera : C:\Program Files
    - set ProgramFiles
- **Modifier la valeur d'une variable:** set NOM_DE_LA_VARIABLE=valeur
  - L'exemple suivant modifie la valeur de JAVA_HOME
    - set JAVA_HOME=C:/programs/jdk1.7

    Pour rappel, on peut afficher la valeur d'une variable dans une ligne de commande en l'entourant de pourcentage : %VARIABLE%. Ce qui nous permet de prendre connaissance de ce bout de code utilisé fréquemment et permettant de modifier la valeur de la variable d'environnement PATH. ///
    - set PATH=%PATH%C:/nouveau/chemin;
- **Supprimer une variable d'environnement**
  - set MA_VARIABLE=

Variables d'environnements

A noter

Ces modifications ne marcheront que dans la console courante. En clair, si vous avez une autre fenêtre de console ouverte à côté : cela ne changera rien pour elle ! Pire, si on ferme la fenêtre où l'on a fait les modifications : celles-ci sont perdues !

Du coup, le mieux c'est de faire un clic droit sur le poste de travail > Propriétés > Onglet Avancés et de cliquer sur « Variables d'environnement » pour gérer ça grâce à l'interface de Windows. Les modifications sont enregistrées, mais il faut quand même recharger toutes les consoles et programmes Windows ouverts pour utiliser ces modifications…

Préparation de l'environnement sous Windows

- Set ORACLE_HOME=E:\APP\oracle\...\dbhome
  - le produit oracle (le chemin d'installation d'oracle)
- Set ORACLE_SID=NCDB
  - là ou on spécifie le nom de la base de données en question
  - Un seul serveur oracle peut contenir plusieurs BDD
  - Cette variable spécifie la BDD qu'on veut utiliser
- Set PATH=E:\APP\oracle\...dbhome\bin
  -

Préparation de l'environnement sous Linux

- export ORACLE_HOME=/u01/app/oracle/product/…/dbhome

- $ export ORACLE_SID=NCDB
  - $ sqlplus / as sysdba
  - SQL> show user;
  - SQL>select name, open_mode from v$database;

Historique

- Oracle Corporation is an American company founded in 1977
  - under the name Consultancy Software Devlopment Laboratories
  - then renamed to Relational Software Inc.
  - In 1979, RSI introduced Oracle V2 as the first commercially available SQL-based RDBMS
  - In 1983, RSI became Oracle Systems Corporation and then Oracle Corporation



Historique (History)

- Founder: Larry Ellison born on 17 août 1944 in New York (United States)
- In 2009 Oracle acquired Sun Microsystems

Oracle Version 3, released in 1983, was the first relational database to run on mainframes , minicomputers, and PCs. The database was written in C, enabling the database to be ported to multiple platforms.

Version 4 introduced multiversion read consistency, Version 5, released in 1985, supported client/server computing and distributed database systems.

Version 6 brough enhancements to disk I/O, row locking, scalability, and backup and recovery also. Version 6 introduced the first version of the PL/SQL langage, a proprietary procedural extension to SQL

Historique (History)

- Oracle 7, released in 1992, introduced PL/SQL stored procedures and triggers.
- Oracle8 Database, released in 1997as the object-relational database, supporting many new data types. Additionally, Oracle8 supported partitioning of a large tables.
- Oracle8i was released in 1999, provided native support for internet protocols and server-side support for java. Oracle 8i (internet) was designed for Internet computing, enabling the database to be deployed in a multitierenvironment.
- Oracle9i Database introduced Oracle RAC in 2001, enabling multiple instances to access a single database simultaneously. Additionally, Oracle XML Database (Oracle XML DB) introduced the ability to store and query XML.

Historique (History)

- Oracle Database 10g introduced grid computing in 2003. A computer architecture that coordinates large number of servers and storage to act as a single large computer
- Oracle 11g, released in 2007, introduced a host of new features that enabled administrators and developpers to adapt quickly to changing business requirements, the key to adaptability is simplifying the information infrastructure by consolidating information and using automation whereverpossible.
- Oracle Database 12c, released in 2013, was designed for the Cloud, featuring a new Multitenant architecture, In-Memory column storage, and support for json documents. Oracle Database 12c helps customers make more efficient use of their IT resources, while continuing to reduce costs and improve service levels for users.

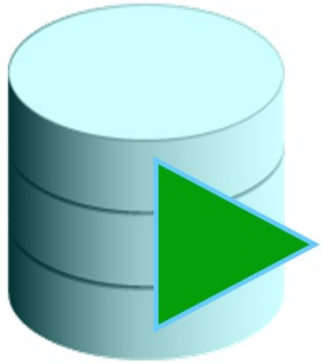
Quelques produits oracle

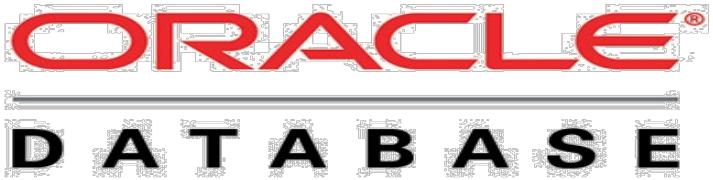
- Oracle WebLogic Server (un serveur d'applications)
- Oracle E-Business Suite (un progiciel de gestion intégré)
- Oracle SQL Developer (un logiciel de manipulation des BD)
- Oracle Database (un système de gestion de base de données)
- …

ORACLE®
E-BUSINESS SUITE

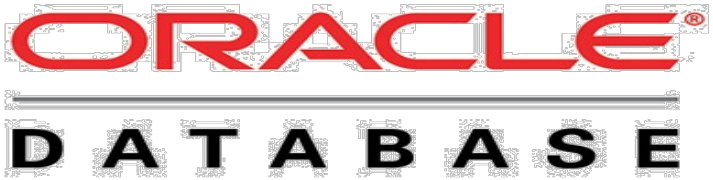
ORACLE®
WEBLOGIC SERVER

ORACLE®
DATABASE





- Système de gestion de base de données relationnel (SGBDR)
- Devenu un SGBDRO à partir de la version 8
- Environnement multiplateforme
- Le produit principal d'Oracle est passé depuis la version 1 à la version 21c en 2022
- Chaque version est commercialisée sous différentes éditions
- Oracle est le leader du marché des SGBDR
- Avec une part de marché allant jusqu'à 48.6% en 2007 selon GARTNER GROUP


ORACLE®
DATABASE

Introduction

❑ Oracle: Vue d'ensemble

➢ Oracle est classé parmi les leaders du marché des SGBDR en 2021 (Gartner Group).

➢ Oracle commercialise différents produits; Oracle Database, Oracle Developer Suite, Oracle Application Server, Oracle Applications, Oracle Collaboration Suite, Oracle Services…

Introduction

❑ Oracle Database

➢ Le produit principal d'Oracle est passé depuis 1977 de la version 1 à la version 21c en 45 ans. Chaque version est commercialisée sous différentes éditions:

- ➢ Enterprise Edition: inclut toutes les fonctionnalités d'Oracle.
- ➢ Standard Edition: basique, destinée aux serveurs à 4 processeurs.
- ➢ Standard Edition One: basique, destinée aux serveurs biprocesseurs.
- ➢ Personal Edition: uniquement sur Windows, destinée aux développeurs.
- ➢ Express Edition: édition gratuite, fonctionne sur des machines à 1 processeur.
- ➢ Lite Edition: destinée aux machines mobiles.

Introduction

❑ Oracle Database Express

- Oracle propose depuis fin 2005 une version totalement gratuite:
  - Oracle database Express Edition 10g
    - Elle limite l taille de la base à 4Go (hors système, temporaire, segments d'annulation et autres fichiers)
    - La SGA ne peut dépasser 1Go et un seul processeur sera utilisé sur les machines qui en disposent de plusieurs
  - Oracle Database Express Edition 11g
  - Présente les mêmes limitations, à part la taille des données utilisateurs qui est portée à 1Go.

Introduction

❑ Oracle Database

	Version.	Release.	Patch.	Sous patch
Exemple:	10.	2.0	1.0.	1.0

- Ex.: 10g première version : 10.0
- Note: Release presque après un an de travail.

- Les niveaux
  - DBA1: L'architecture d'racle, l'administration…
  - DBA2: Buckup, restauration…
  - Tuninig : la performance…

Les SGBDs

MySQL: Open Source

Postgre: Open source

SQL server: Crée par Microsoft

DB2: Crée par IBM

Sybase: Crée par Sybase

Oracle: crée par Oracle

…









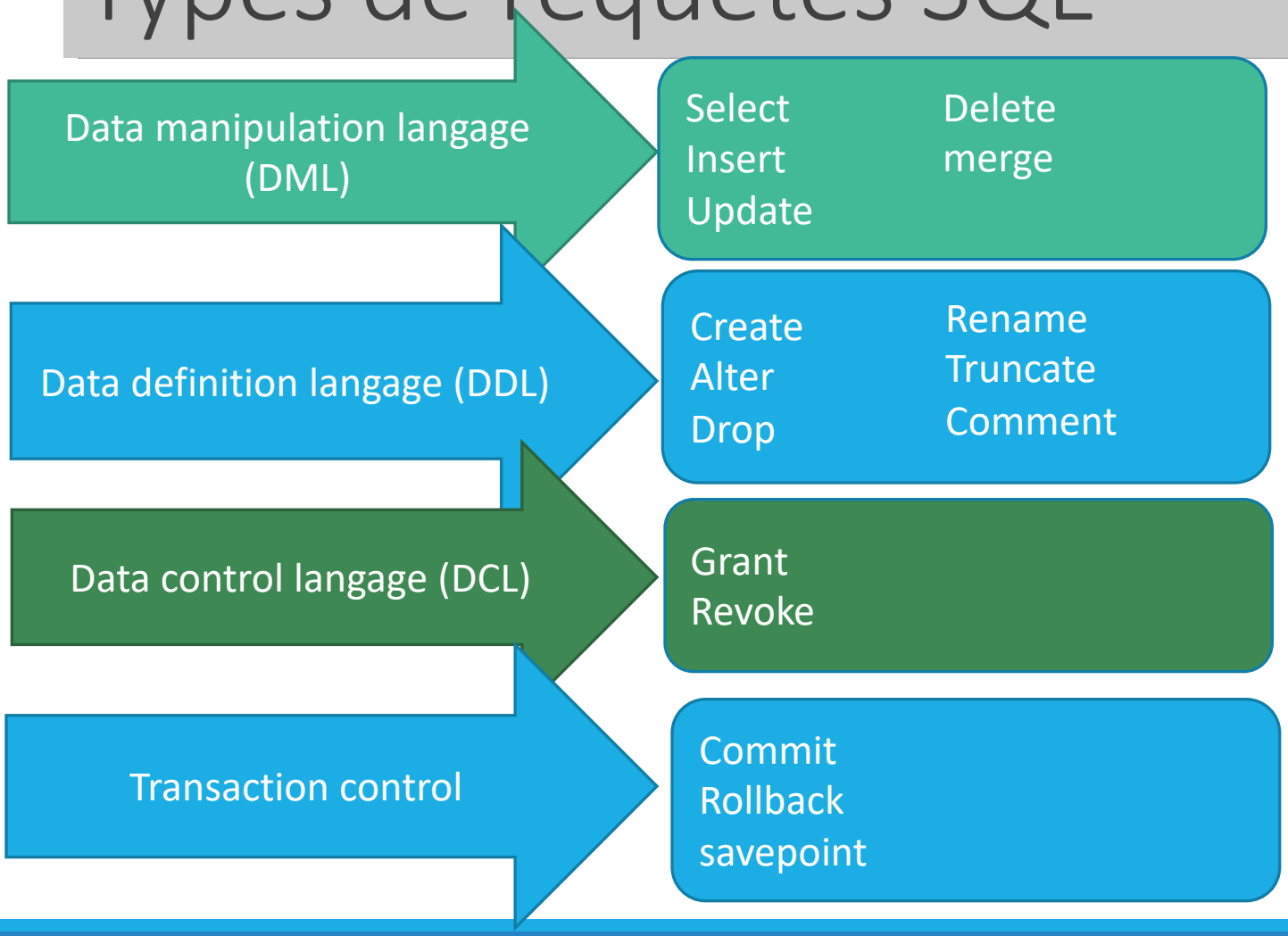




Comparaison des SGBDs

- **SQL Server:**
  - On ne connait pas qu'est-se-qui se passe derrière l'application de Microsoft
  - C'est une application fermée
  - On ne peut utiliser ces produits que sur des plateformes fournies par Microsoft elle-même
  - Pareil pour DB2
- Point fort d'oracle DB
  - Oracle est un produit non fermé
  - Oracle est produit tunnable
  - Oracle est un produit portable

Types de requêtes SQL



Rôles du DBA

Installer les logiciels oracle

- Un serveur, des applications clientes
- En fonction de la configuration système disponible
- Si fonctionnement en réseau: composants réseaux d'oracle

Planifier, créer, manipuler, maintenir les données

Comprendre l'architecture logique et physique d'une BDD oracle

Assurer la sécurité des données

Effectuer des réglages pour optimiser les performances

Allouer des espaces système et planifier des besoins futurs

Sauvegarder et restaurer les données

Assurer le bon fonctionnement du serveur

Le noyau de base d'oracle

- Moteur SQL
- Moteur PL/SQL
- Moteur Java

- Version :

- Select * from PRODUCT_COMPONENT_VERSION ||
- Select * from V$VERSION

Architecture d'Oracle Database

❑ Composantes principales d'un serveur Oracle

1. Base de données

Ensemble de fichiers (sur disque) contenant les données, les informations sur les données, le journal de modifications

2. Instance

Ensemble de processus et de zones mémoires (mémoire centrale) qui permettent de gérer la base de données.

Disque

C’est la partie dans laquelle oracle stocke les données d’une façon permanente.

Exemple de fichiers:

- Fichier d’installation
- Fichier de stockage
- Fichier de paramétrage
- …

Processus en arrière-plan

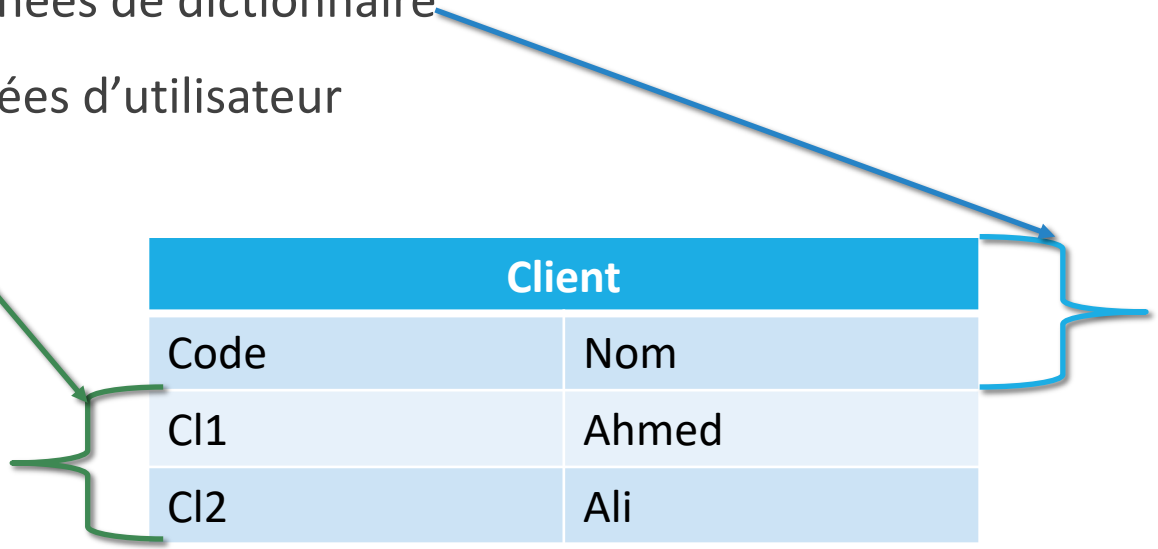
Ce sont des processus qui s'exécutent en arrière-plan, ils font des taches pendant l'exécution des requêtes

Assurent le fonctionnement interne du SGBD Oracle (gestion de la mémoire, écriture dans les fichiers..).

Exemple de Processus en Arrière-Plan: LGWR…

Types de donnée

- Meta data : données de dictionnaire
- Data user: données d'utilisateur
- Exemple:



Client	
Code	Nom
Cl1	Ahmed
Cl2	Ali

Types de donnée

- Donnée de dictionnaire: Meta data
  - Ce sont des données de la structure de la BDD(table, view, champs,…) ou les informations qui identifient les utilisateurs (privilèges …)
  - Elles contiennent toutes les informations de toutes les composantes logiques et physiques de la base de données ainsi que de l'instance
- Données utilisateur: user data
  - Ce sont des données qui sont stockées dans une table crée par un utilisateur

Oracle utilise

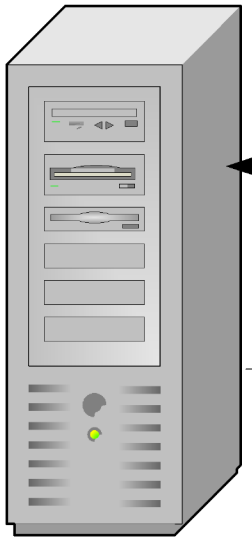
- **Comme tout programmes:**

1. CPU: Traitement et calcul (langage binaire)
2. RAM: stocke les données temporairement
3. Disque: Stocke les données permanant

Alors comment oracle organise ces composants???

Introduction

❑ Interaction avec Oracle Database



Serveur base de données

Interface

Une interface permet à un utilisateur de gérer et de manipuler les données qui sont sur le serveur



Utilisateur

-SQL*Plus

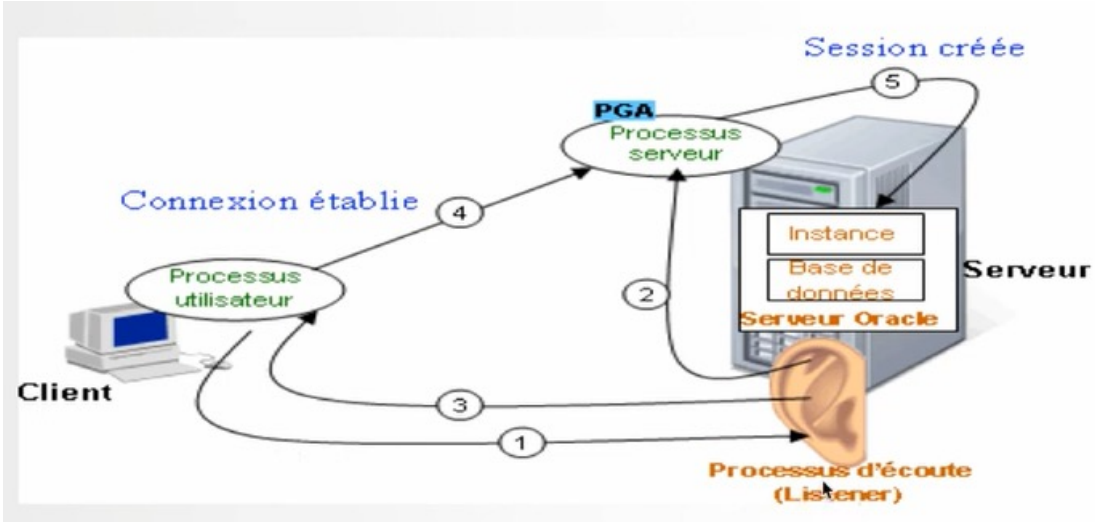
-iSQLPlus

- OEM (DB Control)

- Oracle Discoverer, Oracle Reports, Oracle Forms

- PL/SQL

Architecture générale



1: le client contacte le listener
2: le listener démarre un processus dédié appelé processus serveur
3: le listener envoie un accusé de réception au client avec l'adresse du processus serveur dédié
4: le client établit une connexion avec le processus serveur dédié
5: le processus serveur se connecte à l'instance oracle pour le compte du processus utilisateur

ORACLE DATABASE

Vue générale

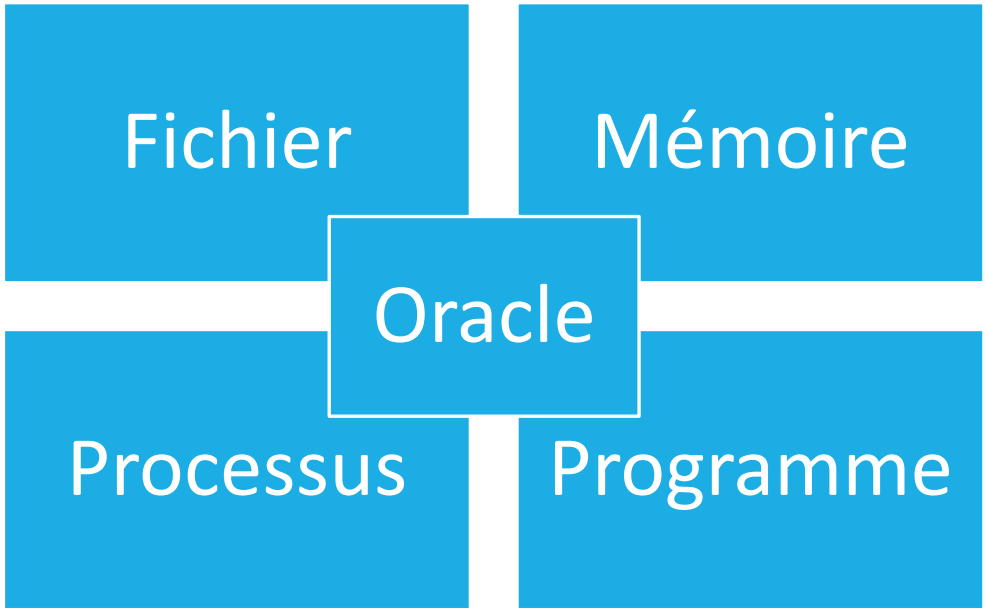
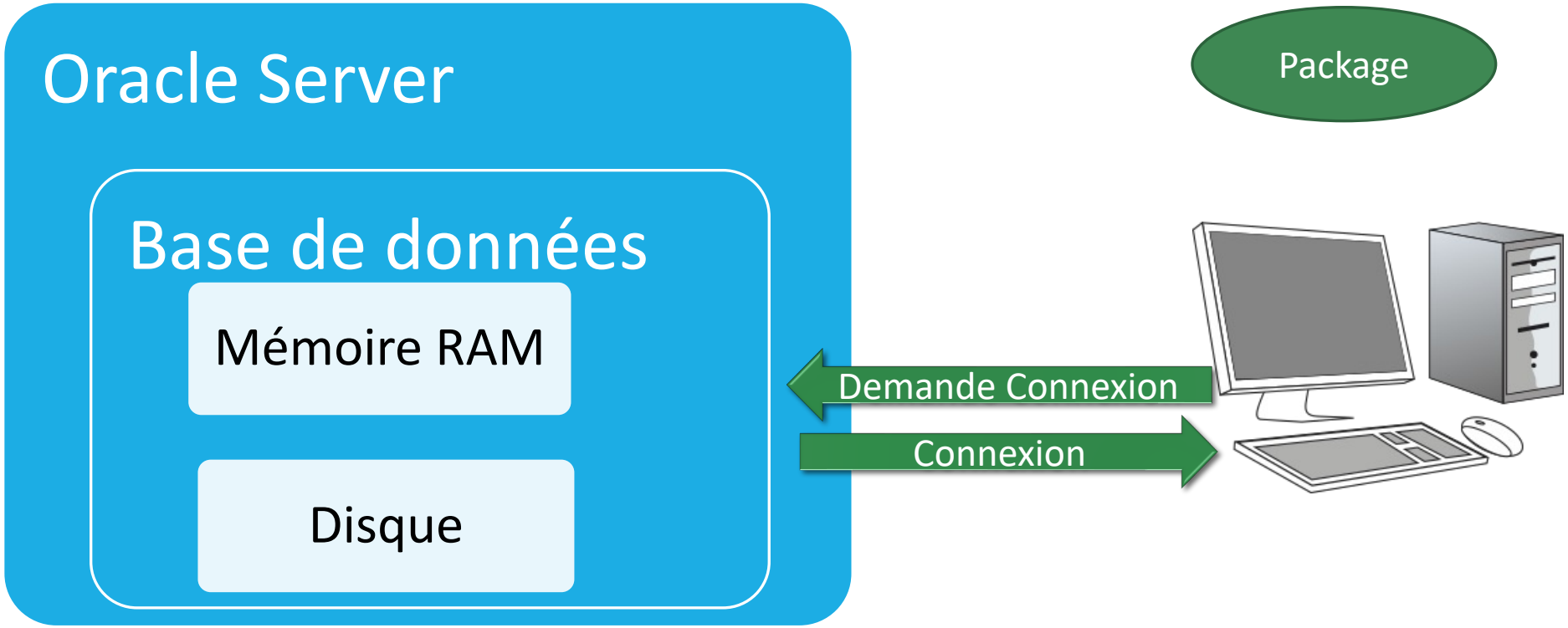


Schéma client et serveur



Le paquet

- Package/ paquet:
  - Adresse IP (ou bien le nom de la machine)
  - Le port (par défaut 1521)
  - Le nom de la base de données (SID)
  - Username/password
  - La requête



Pour Vérification

- Ping NomMachine ou
- Ping IP

Les Moyens de transports

Pour se connecter à une base de données, on a besoins de transférer les données vers le serveur Oracle

Ces Moyens de transport sont des processus qui font le transport d'un paquet entre les composants

Ils transfert les données depuis oracle client vers le serveur de la base de données

Notez que dans oracle on a la notion de moyen de transport

Exemple de moyen de transport entre le client et le serveur d'oracle:

- **Processus client**: créer par oracle
- **ODBC**: installé par défaut sur Windows (il se base aussi sur processus client)
- **JDBC**: driver de Java

Architecture d'Oracle Database

Base de données :

Instance : mémoire + processus

Deux types de configurations du serveur à propos de l'instance:

- Single database instance: 1 to 1
- One to many: par exemple 3 BDD pour 3 instances
- RAC: many to one: plusieurs instances accèdent à une seule BDD ➔ gain de performance +availability (disponibilité).
  - Oracle Real Application Clusters (RAC) permet aux clients d'exécuter une seule base de données Oracle Database sur plusieurs serveurs afin d'optimiser la disponibilité et de permettre une évolutivité horizontale, tout en accédant au stockage partagé. Les sessions utilisateur qui se connectent aux instances Oracle RAC peuvent basculer et relire en toute sécurité les modifications pendant les pannes, sans aucune modification des applications des utilisateurs finaux, ce qui masque l'impact des pannes aux utilisateurs finaux.

Dediacted server

C’est le mode par défaut (l’architecture par défaut)

Une relation un à un est fait entre processus utilisateur et processus serveur (1PS = 1PU)

Donc un processus s’occupe d’un client unique(un seul client à la fois)

Pour les connexions 10000 users

Shared server

- Un processus s'occupe de plusieurs clients
- Réduit l'utilisation de ressources de système
- 1 processus serveur = n processus clients
- La priorité selon la première connexion
- Le processus serveur reçoit les requêtes utilisateurs, les exécutes et renvoie le résultat:
  - en mode dédié, on dédie un PS à chaque utilisateur
  - en mode partagé, on partage un PS pour un ensemble d'utilisateurs

Processus client→Listener

- Processus client:
  - Oracle client déclenche le processus client
  - C'est un moyen de transport
  - Il recherche sur la machine puis le port …
- Le Listener reçoit la requête depuis le processus client et il
  - Ouvre le port
  - Il écoute
  - Il vérifie l'existence de la BD(SID)
  - C'est lui qui assure toutes les ***connections à distance***

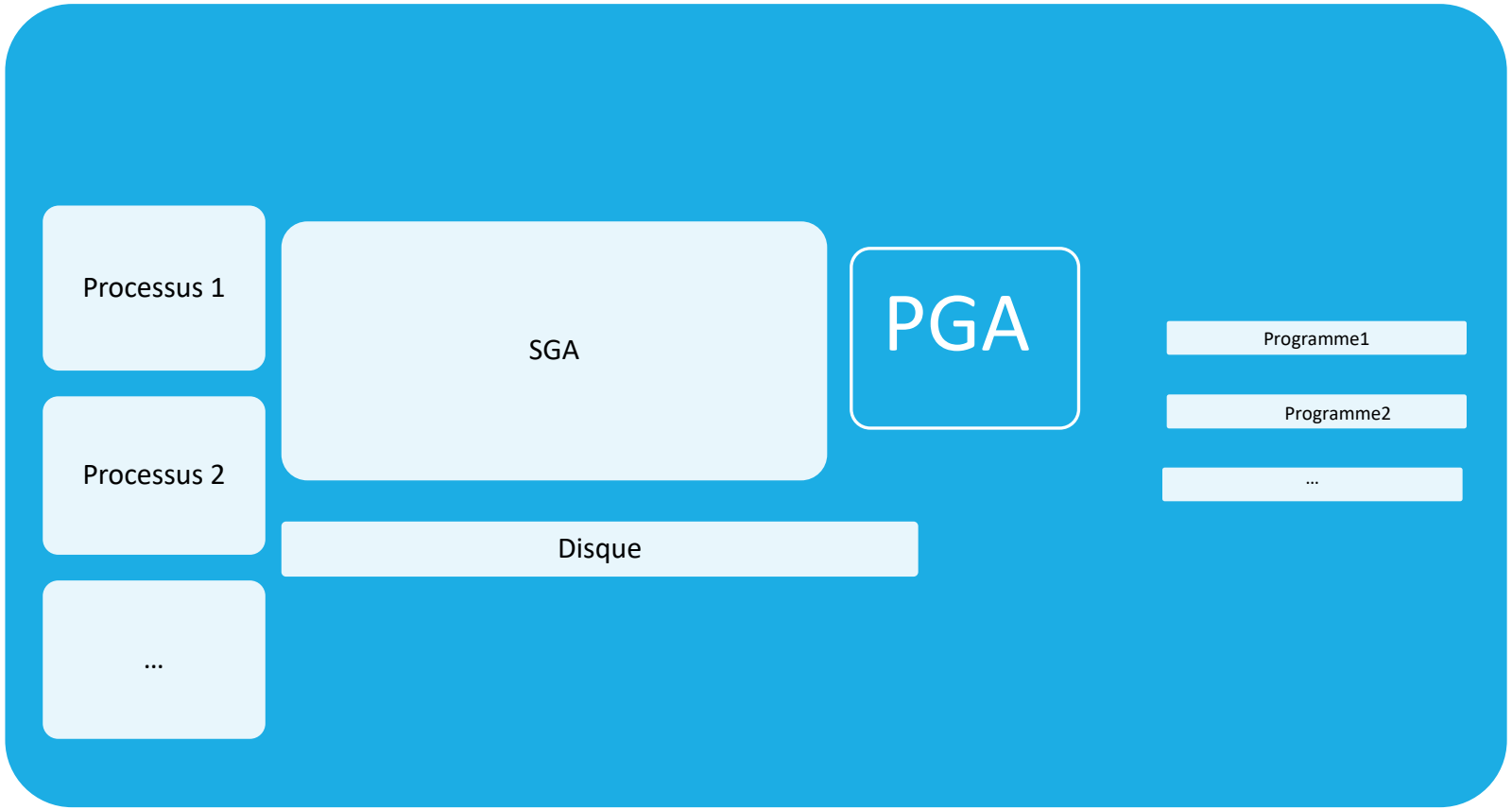
- ***Apres vérification du SID (favorable) le listener informe le processus client, et ce dernier crée un Tunnel de communication (lors de la première connection)***

Processus client

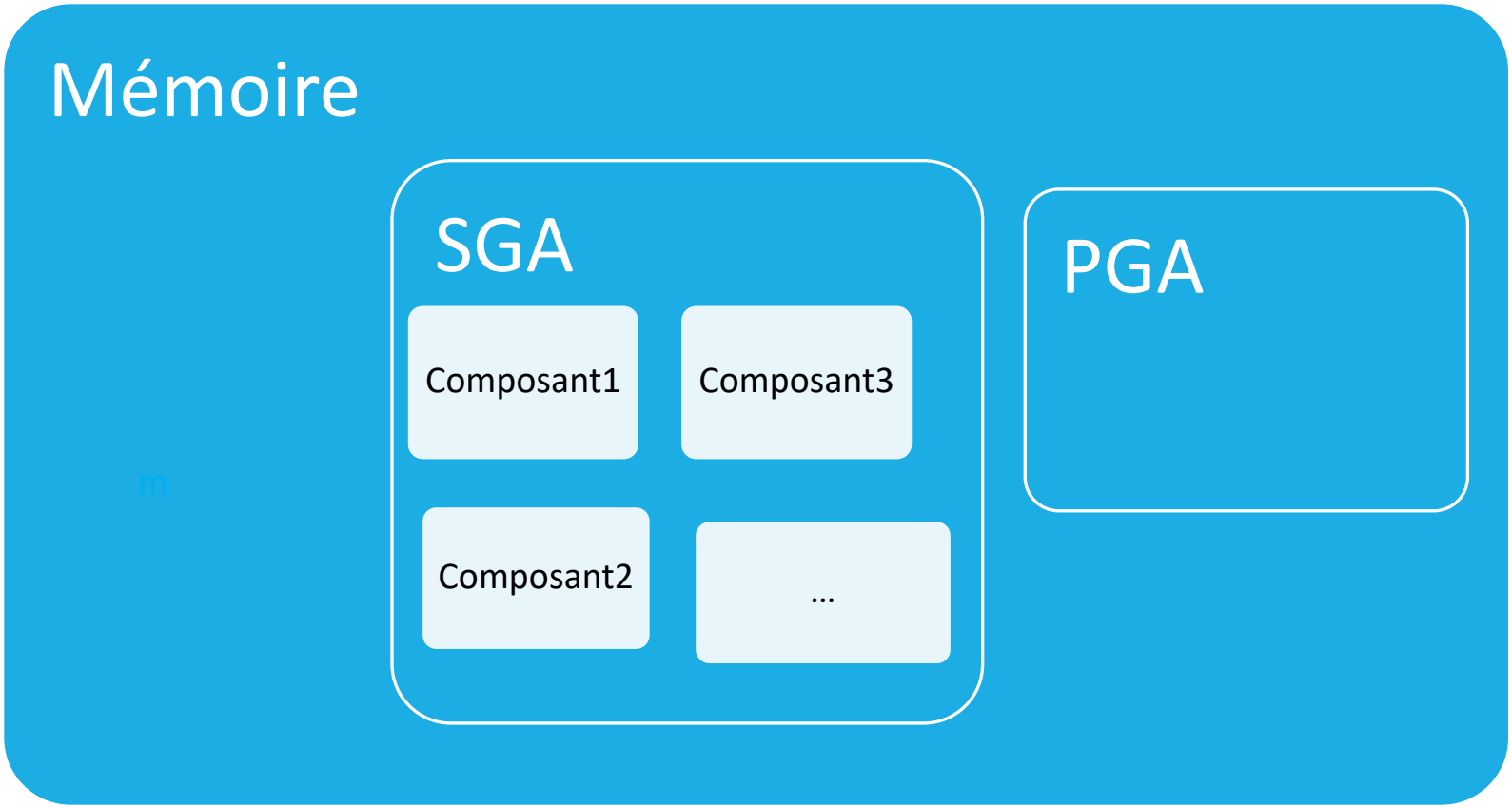
Il crée un Tunnel de communication lors de la première connexion.

➔ Le processus client travail une seule fois et dans la première connexion

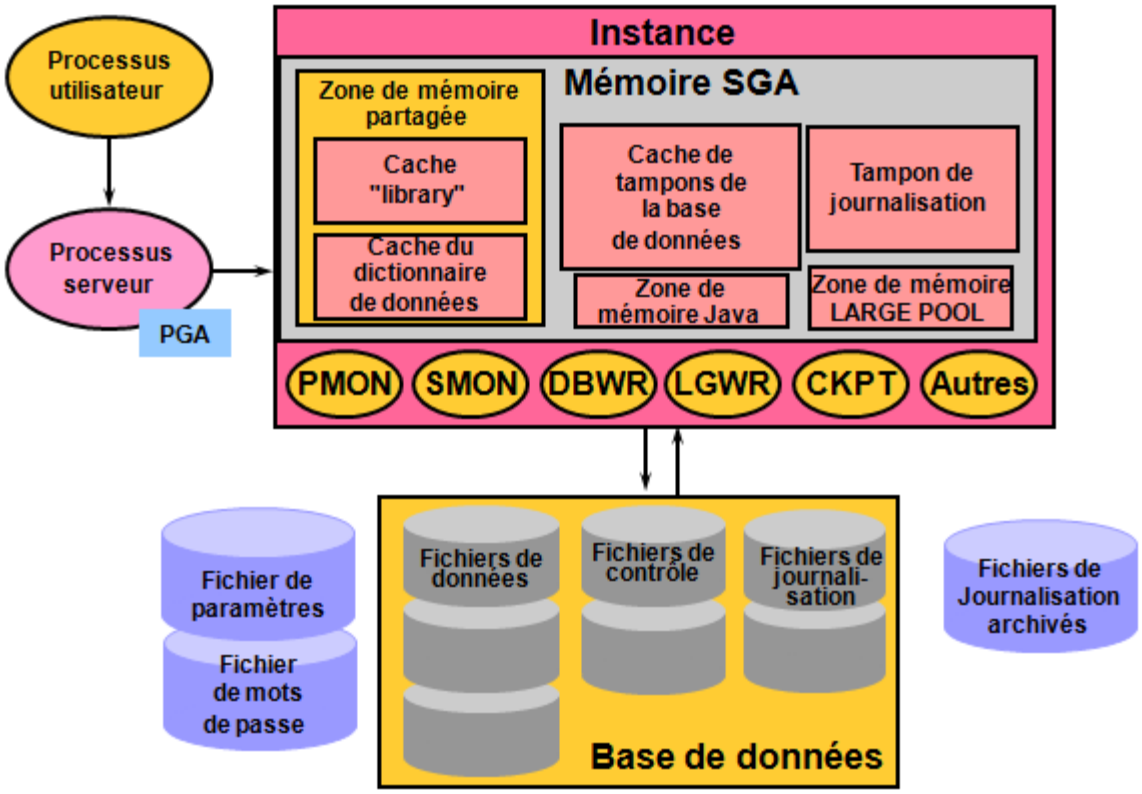
Vue détaillée



SGA et PGA



Architecture d'Oracle Database



Processus serveur

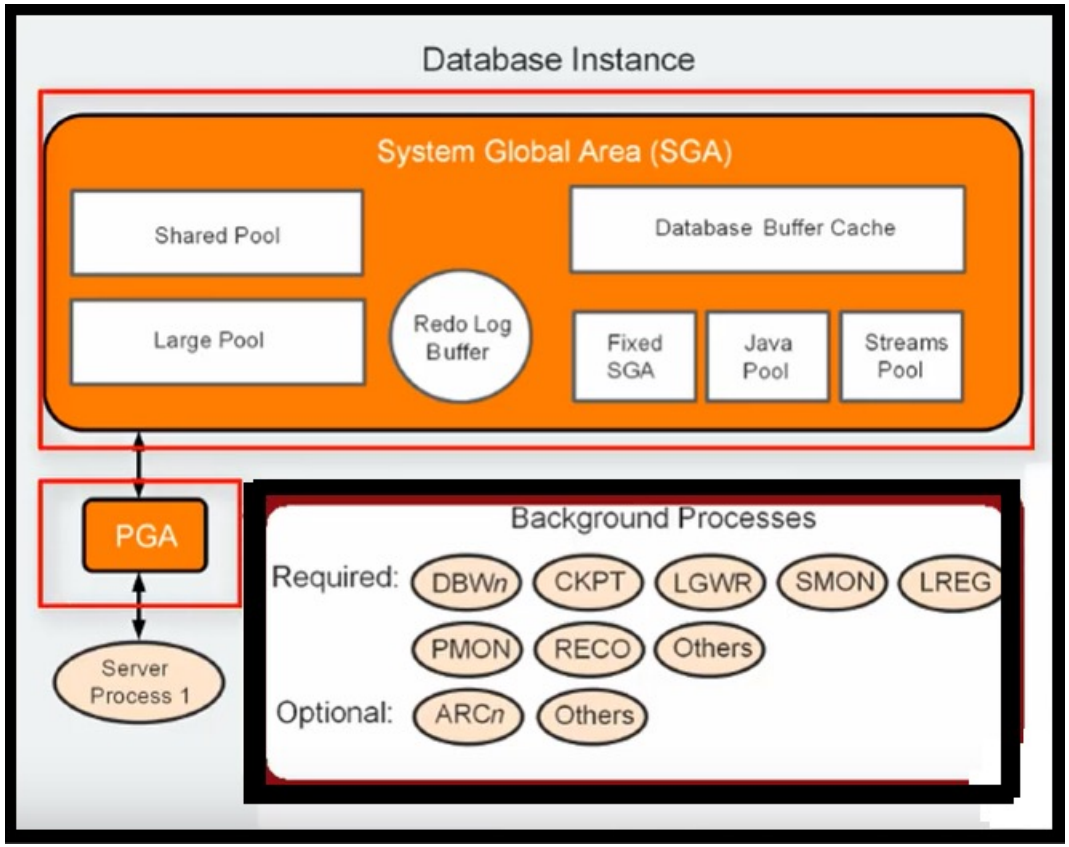
Ses rôles principaux sont:

- traiter la demande du client ➔ quand un client est connecté à une BDD oracle construit un processus serveur (s'il y'a 5 utilisateurs sont connectés, oracle construit 5 processus utilisateurs ➔ dedicated server process 1 to 1) mais si il y'a 10.000 utilisateurs connectés ➔ 10.000 processus serveurs? Et 10.000 PGA ? ➔ shared server process (1 to many)
- Run and parse(syntaxique, sémantique et shared pool check) la requête SQL et établit le plan d'éxécution
- Exécute le code pl/sql, chargement des données à partir du fichier physique dans la zone mémoire (Buffer Cach)
- Renvoie le résultat

Donc j'ai besoin d'une mémoire pour manipuler ces tâche ➔ PGA

PGA est une mémoire pour faire le travail du processus serveur(pour servir le processus serveur)

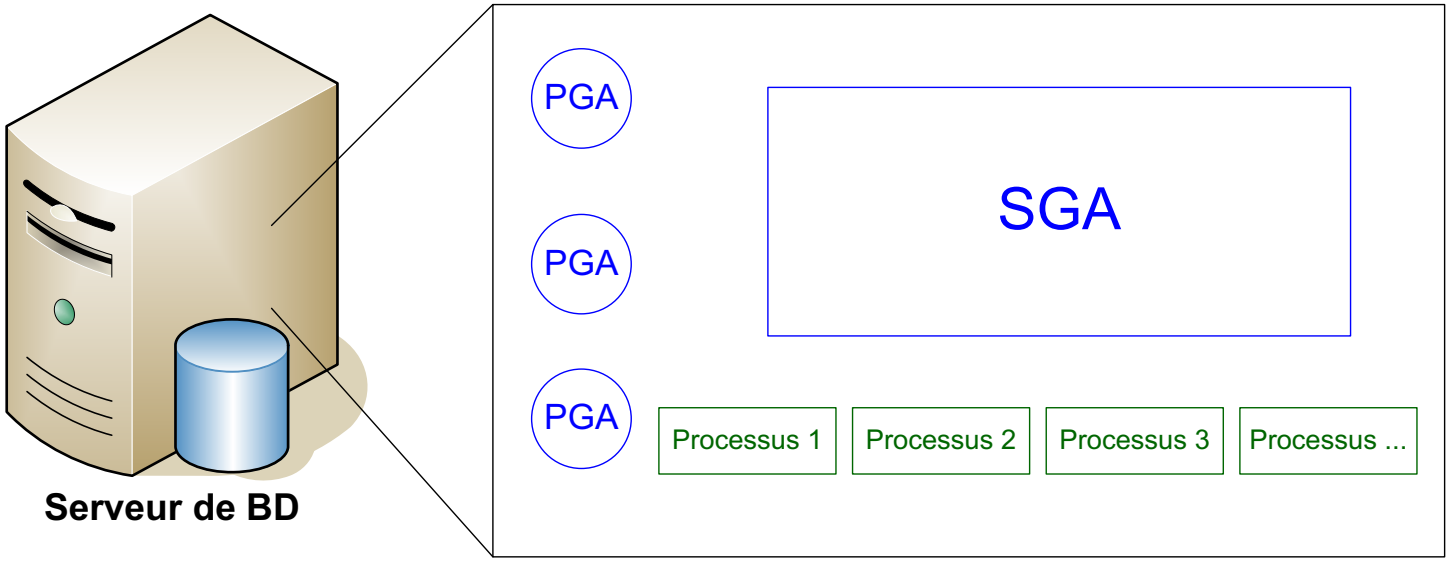
Processus serveur



L'instance

- C'est l'ensemble de structures mémoire et de processus qui assurent la gestion de la base de données.
- Le fichier de paramètres est utilisé pour configurer l'instance lors de son démarrage.
- Une instance ne peut ouvrir qu'une seule base de données.
- Une instance emploie deux zones mémoire principales; la SGA (System Global Area) et la PGA (Program Global Area).

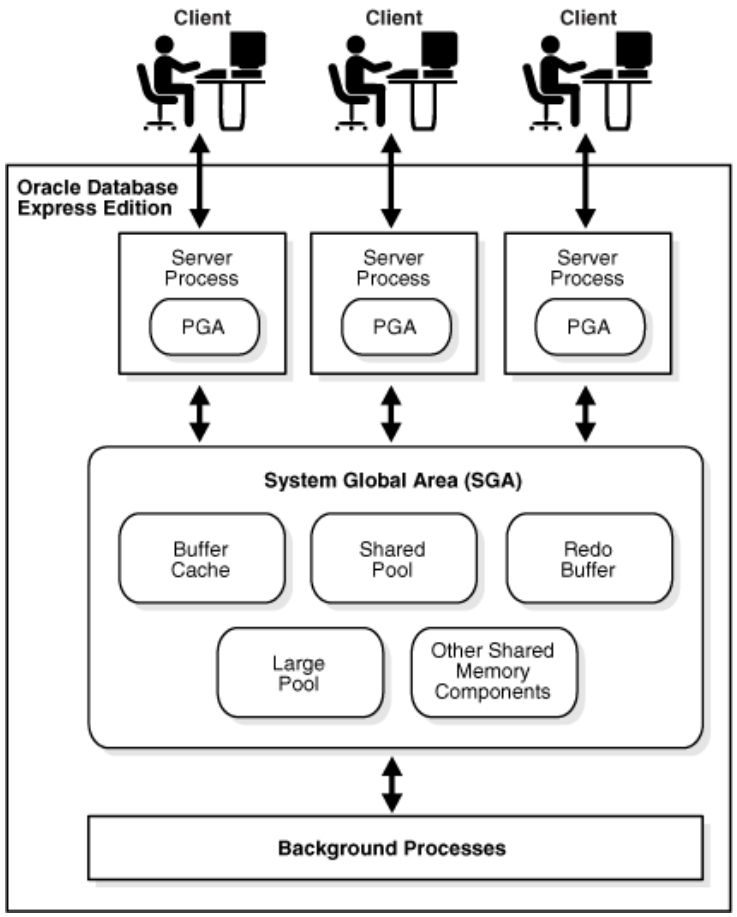
L'instance


PGA
PGA
PGA
SGA
Processus 1
Processus 2
Processus 3
Processus ...
Serveur de BD

La mémoire PGA

- En dehors de la SGA, chaque processus serveur possède une zone mémoire privée appelée PGA (program global area).
- La zone mémoire allouée pour le fonctionnement de chaque processus utilisateur au niveau du serveur
- Lorsque le processus utilisateur se déconnecte (fin de session), le processus serveur associé prend fin et la mémoire PGA est libérée
- Pour un processus serveur la PGA contient:
  - ✓Des informations sur la session
  - ✓Des informations sur le traitement des requêtes de la session
  - ✓Les variables de session

Mémoire PGA



Paramétrage de la PGA:

- PGA est dynamique et est configurée par le système PGA_AGGREGATE_TARGET

Affichage des paramètres de la PGA:

- Select component, current_size from V$MEMORY_DYNAMIC_COMPONENTS;
- Show parameter PGA

- alter system set pga_aggregate_target=100M;

Mémoire PGA

La taille de la PGA est influencée par divers facteurs, tels que la complexité des requêtes, les opérations de tri, et d'autres activités liées à la mémoire dans le cadre de l'exécution de requêtes.

Pour obtenir la taille actuelle de la PGA pour une session particulière, on peut utiliser la requête suivante dans Oracle :

```
SELECT * FROM WHERE 'global memory’;
```

Pour obtenir la taille totale de la PGA pour toutes les sessions:

```
SELECT SUM(VALUE) AS Total_PGA_Size FROM V$PGASTAT WHERE
  'global memory’;
```

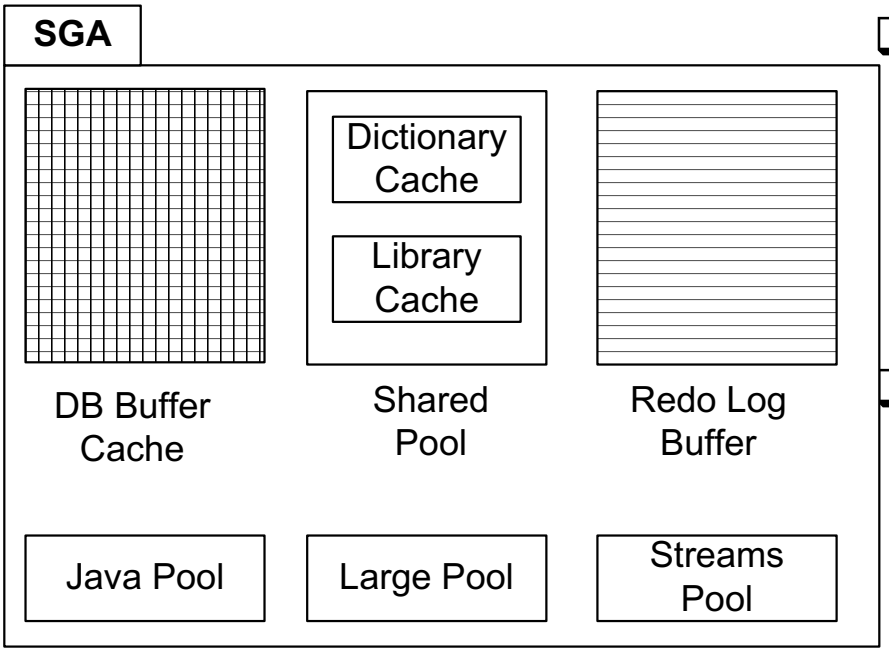
La PGA

- C’est une zone mémoire privée dédiée aux utilisateurs. Elle est créée pour chaque processus serveur.
- Elle stocke des informations spécifiques aux utilisateurs, tel que les variables hôtes, les variables de session, l’état des curseurs utilisés, des informations de tri etc.
- La PGA totale allouée à tous les processus serveur est appelée PGA agrégée.
- Le paramètre `PGA_AGGREGATE_TARGET` définit la taille de la PGA agrégée, c’est Oracle qui se charge de la répartir entre les différentes processus serveur.

La PGA

- La PGA contient les données et les informations de contrôle pour son processus serveur.
- Pour une architecture serveur dédié, elle contient :
  - Une zone de tri (Sort area) : Utilisé pour stocker, si nécessaire, les résultats intermédiaires lors du tri.
  - Des informations de session : Contient les privilèges utilisateurs pour la session en cours.
  - Un état de curseur (Cursor State) : Etat de l'étape dans l'exécution des éventuels curseurs de la session.
    Un curseur est un pointeur sur la mémoire associé à une requête SQL donnée. C'est une zone de travail utilisé pour stocker le résultat de la requête.
  - Un espace de pile (Stack Space) : Contient les variables de session et les arrays

Instance: la SGA



- C'est un espace mémoire partagé par tous les processus de l'instance. Elle est allouée au démarrage de l'instance et libérée lors de son arrêt. Elle inclut 6 zones mémoires (3+3).
- Plus la SGA est grande, plus le serveur est performant. Sa taille maximale est définie via le paramètre SGA_MAX_SIZE

Instance: la SGA

La taille de la SGA sous Oracle 19c dépend de la configuration spécifique de notre instance Oracle et des paramètres que nous avons définis. La SGA est une zone mémoire partagée utilisée par une instance Oracle pour stocker des données et des contrôles globaux.

Pour obtenir la taille actuelle de la SGA, on peut exécuter la requête suivante en tant que sysdba :

```
SELECT * FROM V$SGA;
```

Il est important de noter que la taille de la SGA peut être configurée dans le fichier d'initialisation (init.ora ou spfile.ora) lors du démarrage de l'instance Oracle. Les paramètres clés qui influent sur la taille de la SGA incluent SGA_TARGET, SGA_MAX_SIZE, et d'autres paramètres liés à la mémoire.

Par exemple, on peut avoir la configuration dans le fichier de configuration comme suit :

```
SGA_TARGET=2G
```

Cela signifie que la SGA peut utiliser jusqu’à 2GO de mémoire

Architecture de l'instance

Augmenter la taille de la SGA:

- Select COMPONENT, CURRENT_SIZE from V$MEMORY_DYNAMIC_COMPONENTS;
- SHOW PARAMETER SGA;
- Select * from V$SGA;
- Taille maximale: alter SYSTEM set sga_max_size=tailleM SCOPE=SPFILE;
- Taille maximale: alter SYSTEM set sga_target=tailleM SCOPE=SPFILE;

SGA: Shared Pool

- C’est un espace mémoire dans lequel oracle exécute ses traitements
- Elle est allouée au démarrage de l’instance et libérée lors de son arrêt.
  - Exemple des opérations:
    - Vérification de la syntaxe
    - Exécution des requêtes

Shared Pool

- Elle est composée de deux structures; le **library cache** et le **dictionary cache**. Sa taille est définie via le paramètre **SHARED_POOL_SIZE**.
- Le LC contient pour chaque requête récemment exécutée trois informations:
  - Son texte
  - Sa compilation
  - Son plan d'exécution
- Lorsqu'une requête existe encore dans le LC, Oracle ne perd pas son temps à la ré-exécuter.
- Le DC stocke toutes les informations nécessaires à l'analyse sémantique de la requête (table? Colonnes? Droits d'accès?...)

SGA: Shared Pool

- C'est une zone mémoire partagée (par toutes les connections et toutes les sessions) qui contient des données et des informations sur l'instance
- Elle contient plusieurs zones mémoire: Shared pool, Database Buffer Cache, Redo Log Buffer, Java Pool, Stream Pool …
- Shared Pool:
  - quand un utilisateur envoie une requête: oracle doit savoir si la table existe, si la colonne existe, as-tu les privilèges suffisants, contrainte d'integrité ? Ces informations sont stockées dans le **Data Dictionnary**
    - **Data dictionnary contient les méta data: les tables et les vues , clé primaire …**
    - **Contient la fonction result cache ➔ le résultat d'une fonction**
    - **SQL result cache ➔ le résultat d'une requête sera stocké dans le shared pool. Le problème ici c'est trop d'enregistrer tous les résultats des requête ici, donc on utilise un paramètre dans la base de données qui s'appelle RESULT_CACHE_MODE qui prend soit 'force' soit 'manual'. Il doit être 'manual' : SELECT /*+RESULT_CACHE*/…**

DB Buffer Cache

- Il stocke les blocs de données les plus récemment utilisés.
- Lorsqu'une requête est exécutée, Oracle vérifie d'abord si les blocs de données à ramener ne soient pas déjà chargés dans le DB buffer.
- Si les blocs à renvoyer n'existent pas dans le DB buffer cache, Oracle les y stocke selon l'algorithme LRU (et libère d'autres blocs s'il le faut).
- La taille du DB Buffer Cache est définie via le paramètre `DB_CACHE_SIZE`
- Types de blocs…

DB Buffer Cache

Ci-dessous la requete pour vérifier la taille actuelle du DB Buffer Cache dans Oracle à l'aide de la vue V$BUFFER_POOL :

```
SELECT NAME, BLOCK_SIZE, BYTES FROM V$BUFFER_POOL WHERE NAME = 'DEFAULT';
```

La taille sera indiquée dans la colonne BYTES.

Il est important de noter que la taille du DB Buffer Cache peut être configurée statiquement (fixe) ou de manière dynamique en fonction de la gestion automatique de la mémoire (AMM). Dans le cas de l'AMM, on spécifie une taille cible globale pour la SGA, et Oracle alloue dynamiquement l'espace entre les différents composants, y compris le DB Buffer Cache.

Redo Log Buffer

- Contient les informations sur les transactions exécutées.
- Le Redo Log Buffer est écrit de manière séquentielle (les transactions y sont mélangées) et cyclique (taille limitée de la zone mémoire oblige).
- Chaque modification correspond à une entrée Redo. Cette dernière est constituée de plusieurs vecteurs, chacun correspond à un bloc de données modifié (ancienne + nouvelle valeur).
- Le contenu du Redo Log Buffer est écrit dans les fichiers journaux.
- La taille du Redo Log Buffer est définie via le paramètre `LOG_BUFFER`

Redo Log Buffer

La taille du Redo Log Buffer dans Oracle est déterminée par le paramètre système LOG_BUFFER. Le Redo Log Buffer est une zone mémoire de la SGA utilisée pour stocker temporairement des informations sur les modifications de la base de données avant qu'elles ne soient écrites dans les Redo Log Files sur le disque.
La taille du Redo Log Buffer est spécifiée en octets dans le paramètre LOG_BUFFER.
La requete suivante vérifie la taille actuelle du Redo Log Buffer à l'aide de la vue dynamique V$SGA :

```
SELECT NAME, BYTES FROM V$SGA WHERE NAME = 'Redo Buffers';
```

La taille sera indiquée dans la colonne BYTES.
on peut également consulter le paramètre système directement avec la requête suivante :

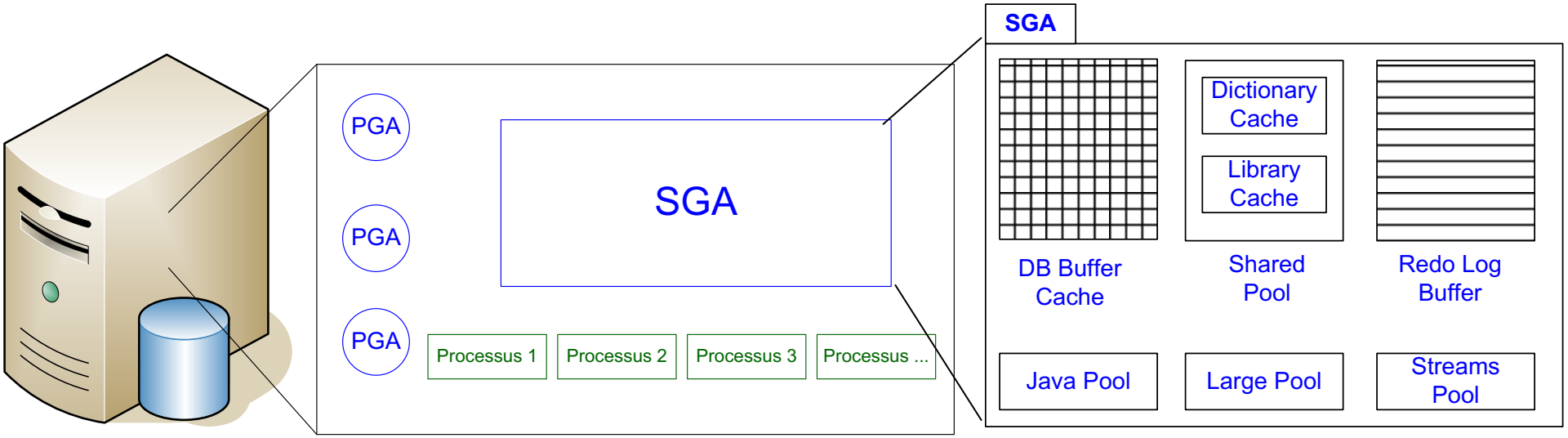
```
SHOW PARAMETER LOG_BUFFER;
```

La taille du Redo Log Buffer est importante car elle affecte la capacité du système à gérer les modifications et les transactions. Une taille de buffer trop petite peut entraîner des performances médiocres, tandis qu'une taille trop grande peut entraîner un gaspillage de mémoire. Donc à vous de gérer.

Autres zones mémoire

- Le JAVA Pool : Stocke les objets et applications Java les plus récemment utilisés lorsque la machine virtuelle Java JVM optionnelle est utilisée. Paramètre `JAVA_POOL_SIZE`.
- Le Large Pool : Stocke des données lors de l'exécution d'opérations volumineuses. Il est utilisé dans le cas d'une configuration serveur partagé. Paramètre `LARGE_POOL_SIZE`.
- Le Streams Pool : C'est le cache des données relatives à la queue de messages utilisées par Oracle. Paramètre `STREAMS_POOL_SIZE`.
- A noter que dans la SGA, il y a une petite zone mémoire appelée SGA fixe. Elle inclut des infos sur l'état de la base, l'instance, les verrous…
- Les vues `V$SGA` et `V$SGA_DYNAMIC_COMPONENTS`.

Récapitulatif SGA



SERVEUR BD =
Instance
+
BD (fichiers)

INSTANCE =
Zones mémoires
+
Processus

ZONES MÉMOIRES
=
3 + 3

Connexion et session

Lorsque le client est lié au serveur, on parle d'une connexion.

- Session:
  - Chaque client, après la première connexion il crée une session dans la BDD. (Lorsque l'utilisateur s'identifie, il ouvre une session)
  - La session est identifiée par : SID et serial SERIAL#
  - Plusieurs sessions peuvent être ouvertes en même temps
  - Elles ne doivent pas dépasser un nombre de paramètres(un nombre maximal de sessions)
  - Le PMON supprime la session après déconnexion
  - Le DBA peut supprimer toute session suivant ses souhaits

Les processus

- Ils permettent une interaction entre les différentes composantes du serveur ainsi qu'avec les utilisateurs.

- Il existe trois types de processus

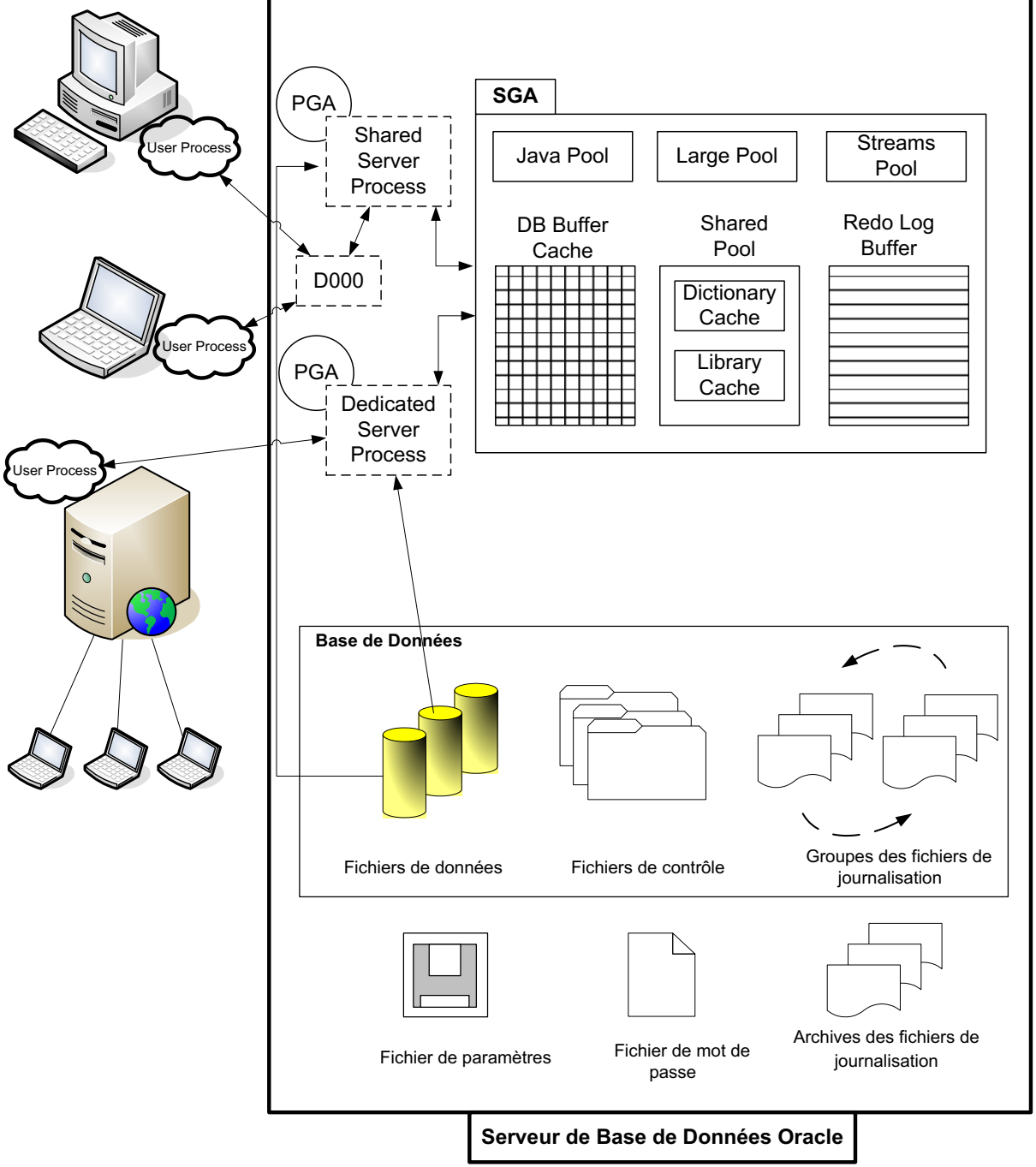
1. Les processus utilisateur
2. Les processus serveur
3. Les processus d'arrière-plan

Les processus utilisateur et serveur

- Le processus utilisateur s'exécute au niveau client.
- Le processus serveur s'exécute au niveau serveur BD.
- L'interaction entre le serveur et les clients se fait en réalité grâce à ces deux processus, chacun de son côté.
- Lorsque le client est lié au serveur, on parle d'une connexion.
- Lorsque l'utilisateur s'identifie, il ouvre une session.
- Plusieurs sessions peuvent être ouvertes en même temps. Elles ne doivent pas dépasser la valeur du paramètre `SESSIONS`.

Les processus utilisateur et serveur

- Le processus serveur (PS) reçoit les requêtes utilisateur, les exécute et renvoie le résultat.
- Un serveur BD est soit en mode DÉDIÉ, soit en mode PARTAGÉ.
- En mode dédié, on dédie un PS à chaque utilisateur.
- En mode partagé, on partage un PS pour un ensemble d'utilisateurs.
- Le mode par défaut est le mode dédié. Pour activer le mode partagé, il faut modifier la valeur de `SHARED_SERVERS` (par défaut =0).
- La valeur de `SHARED_SERVERS` indique le nombre de processus serveur partagés.
- Rappelons qu'une zone de mémoire PGA est allouée à chaque PS.


User Process
User Process
User Process
PGA
Shared Server Process
D000
PGA
Dedicated Server Process
SGA
Java Pool
Large Pool
Streams Pool
DB Buffer Cache
Shared Pool
Redo Log Buffer
Dictionary Cache
Library Cache
Base de Données
Fichiers de données
Fichiers de contrôle
Groupes des fichiers de journalisation
Fichier de paramètres
Fichier de mot de passe
Archives des fichiers de journalisation
Serveur de Base de Données Oracle

Les processus d'arrière-plan

- Assurent le bon fonctionnement du serveur.
- Maximisent la performance du serveur.
- Démarrent avec ou après (sur demande) le démarrage de l'instance.
- Certains peuvent être exécutés en *n* exemplaires.

Le DB Writer (DBW*n*)

Le DB Writer (DBWn), qui signifie Database Writer, est un processus fondamental dans l'architecture d'Oracle Database. Son rôle principal est de transférer des blocs de données depuis la mémoire cache (Buffer Cache) vers les fichiers de données sur le disque, en assurant ainsi la cohérence des données entre la mémoire et le stockage permanent.
Ci-dessous quelques-unes des fonctions et responsabilités clés du DB Writer :

1.Écriture des Blocs Sales sur le Disque : Lorsqu'un utilisateur exécute une requête de modification (INSERT, UPDATE, DELETE), les changements sont d'abord effectués en mémoire dans la Buffer Cache. Les blocs de données modifiés, appelés "blocs sales", doivent être écrits sur le disque pour refléter les modifications de manière permanente. C'est le rôle du DB Writer.

2.Contrôle des Transactions : Le DB Writer joue un rôle crucial dans la gestion des transactions. Il garantit que les transactions commitées ont été écrites sur le disque, préservant ainsi la durabilité des transactions.

Le DB Writer (DBW*n*)

1.Optimisation de l'Ecriture sur le Disque : Plutôt que d'écrire chaque bloc modifié immédiatement sur le disque, le DB Writer utilise des algorithmes sophistiqués pour décider quand et quels blocs écrire. Il cherche à minimiser l'impact sur les performances en regroupant les écritures autant que possible.

2.Gestion des Priorités : Le DB Writer gère les priorités d'écriture en fonction des besoins du système. Certains blocs peuvent être plus critiques à écrire que d'autres, et le DB Writer s'ajuste en conséquence.

3.Contrôle de la Fréquence des Ecritures : La fréquence à laquelle le DB Writer écrit les blocs dépend de plusieurs facteurs, y compris la charge système, la configuration des paramètres Oracle, et la stratégie d'écriture configurée.

L'architecture d'Oracle utilise généralement plusieurs instances du DB Writer, connues sous le nom de DBWn (où n est un nombre unique pour chaque instance). Cela permet de paralléliser l'écriture sur le disque pour améliorer les performances.

En résumé, le DB Writer est essentiel pour garantir la cohérence des données entre la mémoire et le stockage permanent, tout en optimisant les performances d'écriture sur le disque.

Le DB Writer (DBW*n*)

- Nettoie le DB Buffer Cache des blocs « dirty » les moins récemment utilisés en les écrivant sur les fichiers de données.
  - Lors de la création de plan d'execution, l'Optimizer a besoin de savoir les METADATA
  - Il recherche(DBW) dans le DC, s'il ne trouve pas les metadata qu'il veut, il demande à DBWR de récupérer les METADATA depuis le fichier DBF
- Le paramètre qui détermine ce nombre `DB_WRITER_PROCESSES`.

L'écriture des blocs modifiés est différée: pas forcément après la confirmation (COMMIT) de la transaction en question

BDWR

C’est un programme qui se déclenche si les informations nécessaires ne sont pas dans le DC

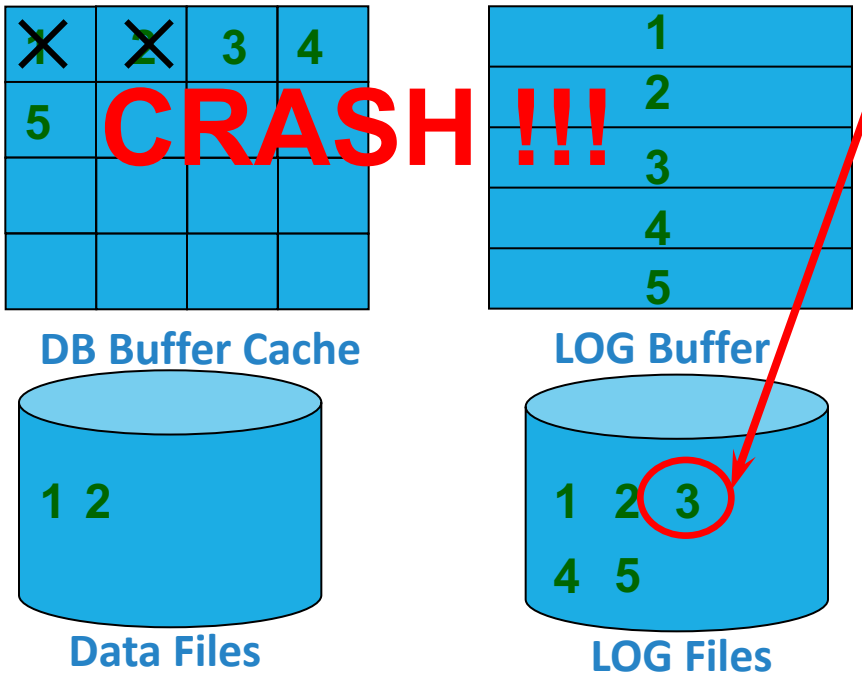
- Il copie les données dans le DC depuis les fichiers DBF
- Il copie juste les données nécessaires, pour la requête en question
- Ce processus est chargé du transfert des blocks de données modifiés du buffer cache dans le disque
- Il est aussi responsable de la vérification en permanence du nombre de block libres dans le buffer cache afin de laisser assez de place disponible pour l’écriture des données dans le buffer. (il utilise aussi PMON avec LRU)

Le DB Writer (DBW*n*)

- Un bloc est dit « dirty » si son contenu a changé. Oracle n ’autorise pas de nouvelles données à prendre cette place, tant que ce contenu n’a pas été écrit sur disque.
- Le DBWn se déclenche (en écrivant les blocs dirty les moins récemment utilisées sur le disque) par ces deux événements:
  1. Le PS ne trouve pas de tampons « clean » après avoir scanné un nombre seuil de tampons.
  2. Après une certaine période pour faire avancer le point de reprise. Le point de reprise est la position dans les fichiers de journaux à partir de laquelle l’instance est récupérable. Cette position est déterminée par le plus ancien tampon dirty dans le cache de données

Le DB Writer (DBW*n*)

- Le DBW*n* se déclenche (en écrivant les blocs dirty les moins récemment utilisées sur le disque) par ces deux événements:
  1. Le PS ne trouve pas de tampons « clean » après avoir scanné un nombre seuil de tampons.
  2. Après une certaine période pour faire avancer le point de reprise.




```
1. Insert…;
2. Update…;
```
Le DBW est déclenché !
```
3. Update…;
commit;
4. Insert…;
5. Update…;
commit;
```

Le point de reprise est la position dans les fichiers journaux à partir de laquelle les données sont récupérables = le plus ancien tampon dirty du DB cache.

Le DB Writer (DBW*n*)

- Le DBW*n* ne se déclenche pas à la suite d'un COMMIT. Il faut donc savoir qu'on peut se trouver dans l'une de ces deux situations:

1. Le DBW*n* écrit des modifications non-confirmés (non commités) dans les fichiers de données. On dit qu'il anticipe.

2. Des modifications confirmées ne sont pas écrites sur les fichiers de données, et restent en mémoire centrale (dans le DB Buffer Cache).

Le LOG Writer (LGWR)

- Le LGWR écrit le contenu du Redo Log Buffer sur les fichiers journaux pour libérer de l'espace au PS.
- Le LGWR est déclenché par :

1. COMMIT → écriture des entrés redo relatives à la transaction confirmée.
2. Rollback
3. Un switch du log file
4. Après un délai (environ toutes les 3 secondes par défaut)
5. Quand le Redo Log Buffer est plein au tiers
6. Lors d'un checkpoint
7. Le Log Buffer Cache se remplit jusqu'à un mégaoctet (1Mo).
8. quand le DBW*n* écrit sur les fichiers de données des blocs dirty.

Le System Monitor (SMON)

- Ou le processus de monitoring

1. En action lors d'un **arrêt brutal** de la base de données:
   - Il lit les informations contenues dans le segment UNDO(contient les données en attente de validation) puis les annule : **ROLLBACK**
   - SMON récupère dans les fichiers REDO LOG les **modifications validées mais pas encore écrit** dans les fichiers de données et les **insère**: ROLLFORWARD
2. Le SMON en action lors d'un fonctionnement **normal** de la base de données:
   - SMON **surveille** l'activité de la base de données
   - SMON **libère toutes les ressources** de la base de données (segments temporaires).

Le System Monitor (SMON)

- Donc le SMON:
  - SMON corrige les plantages de l'instance
  - Vérifie la synchronisation des données
  - Rejouer les transactions qui se trouves dans le REDO LOG BUFFER
  - Valide les transactions valides
  - Annule les transactions non validées
  - Libère toutes les ressources de la BDD

Le Process Monitor (PMON)

- Ou le processus de monitoring
- En action lors d'un **échec d'un processus utilisateur**
  - Il annule la transaction (**ROLLBACK**)
  - PMON **nettoie le cache de tampons** de la base de données
  - PMON **libère** les **zones mémoire allouées** et **annule** les **ressources affectées aux threads** de la transaction.
  - **Analyse et détecte les processus utilisateurs**
- Son rôle c'est le nettoyage de tous les composants de la BDD, exemple:
  1. **Détection du plantage des processus utilisateurs**.
  2. **Supprime les sessions directement après la déconnexion**
  3. **Enlève les plans d'exécutions qui ne sont pas choisis par l'optimizer**
  4. **Détection du plantage des processus utilisateurs**

Le Process Monitor (PMON)

- Ce processus se charge notamment de **libérer toutes les ressources** acquises par le **processus client**, lorsque celui-ci se termine
- Il se charge de **nettoyer** la zone **SGA**
- Il est aussi chargé de **surveiller** les **processus serveurs**: si l'un des deux s'arrête anormalement, le PMON se chargera de **libérer les ressources de ce processus** et de le **relancer**
- Envoie un ordre de **rollback**
- Elle fait le **releaselock**
  - Exemple: si on fait un update pour n lignes sans faire un commit just après. Ces n ligne sont verrouillées(sur les autres users, mais ils sont consultables dans leur état avant modification (read consistency)) ➔
    - ✓ PMON déverrouille ces ligne (**release lock**)
    - ✓ PMON nettoye la mémoire (**cleanup**)

différence entre le PMON et SMON

PMON (Process Monitor) et SMON (System Monitor) sont deux processus fondamentaux dans l'architecture d'Oracle Database, mais ils ont des responsabilités distinctes dans la gestion et la maintenance du système. Voici les principales différences entre PMON et SMON :

- **Responsabilités Principales :**
  1. **PMON (Process Monitor) :** Surveille et gère les processus serveur associés aux sessions utilisateur. Il nettoie les ressources liées aux sessions terminées de manière anormale, récupère les transactions en attente, et maintient la cohérence des sessions utilisateur.
  2. **SMON (System Monitor) :** Gère la maintenance système globale. Il nettoie les ressources inutilisées, valide ou annule les transactions en attente, nettoie les blocs orphelins, répare les blocs de données corrompus, et effectue diverses tâches de maintenance pour garantir la stabilité du système.
- **Gestion des Sessions :**
  1. **PMON :** Gère les sessions utilisateur, s'assure de leur cohérence, nettoie les sessions inactives, et libère les ressources associées.
  2. **SMON :** Ne gère pas directement les sessions utilisateur mais s'occupe de la maintenance systémique qui peut affecter les sessions, comme la validation ou l'annulation de transactions en attente.

différence entre le PMON et SMON

- **Nettoyage des Ressources :**
  1. **PMON :** Nettoie principalement les ressources associées aux sessions utilisateur, comme les verrous et les curseurs.
  2. **SMON :** Nettoie les ressources inutilisées au niveau systémique, y compris l'espace mémoire, les transactions non validées, et les blocs de données orphelins.
- **Gestion des Transactions en Attente :**
  1. **PMON :** Tente de récupérer les transactions en attente associées aux sessions utilisateur.
  2. **SMON :** Gère la validation ou l'annulation des transactions en attente au niveau systémique.
- **Réparation des Blocs Corrompus :**
  1. **PMON :** N'est pas directement impliqué dans la réparation des blocs corrompus.
  2. **SMON :** Peut être impliqué dans la réparation des blocs de données corrompus au niveau systémique.

En résumé, PMON se concentre sur la gestion des sessions utilisateur et la cohérence des transactions, tandis que SMON gère la maintenance systémique, nettoie les ressources inutilisées, et s'occupe de la cohérence globale du système. Les deux processus travaillent ensemble pour assurer la stabilité et la performance d'Oracle Database.

LRU

LRU: Least Recently Used

- Cet algorithme permet d'identifier le dernier élément dans le composant mémoire non utilisé, si la mémoire de composant est pleine
- Utilisé dans tous les composants de la SGA
- PMON supprime les éléments mentionnés par LRU

Redo Log

- Ou bien fichier de journalisation/ log file
- C’est un fichier dans le disque, pour sauvegarder physiquement les logs
- Log: qui a fait quoi?

Redo Log

- Ce sont des fichiers qui permettent la restauration en cas de panne de la BDD
- Ils stockent les données en binaire, donc on ne peut pas les modifier
- A BDD sera stoppée si un fichier est supprimé

Redo Log

- Oracle utilise un ensemble de Redo Log files
- Ces fichiers mémorisent toutes les transactions effectuées par les utilisateurs
- On les appelle aussi fichiers de journalisations
- Ils représentent un historique complet de toutes les commandes (amenant une modification) passées sur la BDD
- L'utilité de tels fichiers est de pouvoir reconstruire les transactions de la BDD dans leur ordre adéquat, si une panne apparait

LGWR

LOG Writer

C’est un processus en arrière-plan se déclenche pour copier les données depuis le log buffer à redo log

C’est le processus qui écrit le contenu des buffers dans les fichiers de journaux pour libérer de l’espace au PS

Groups –Redo log

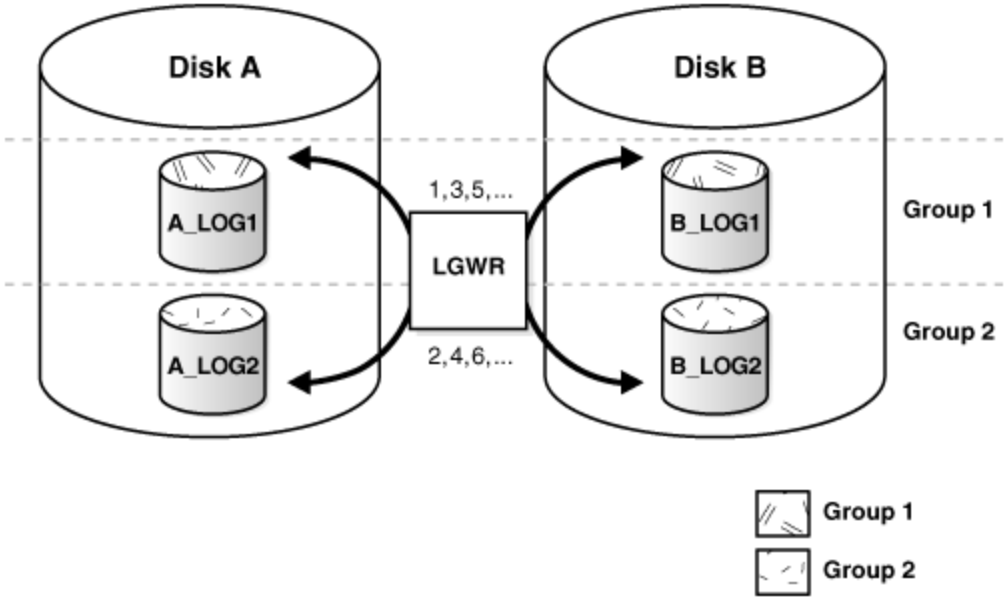
- Oracle a divisé les fichiers log sous forme de groupes
- Un groupe est une organisation logique, et n'existe pas dans le disque
- Il l'utilise pour organiser les fichiers redo log

Groups –Redo log

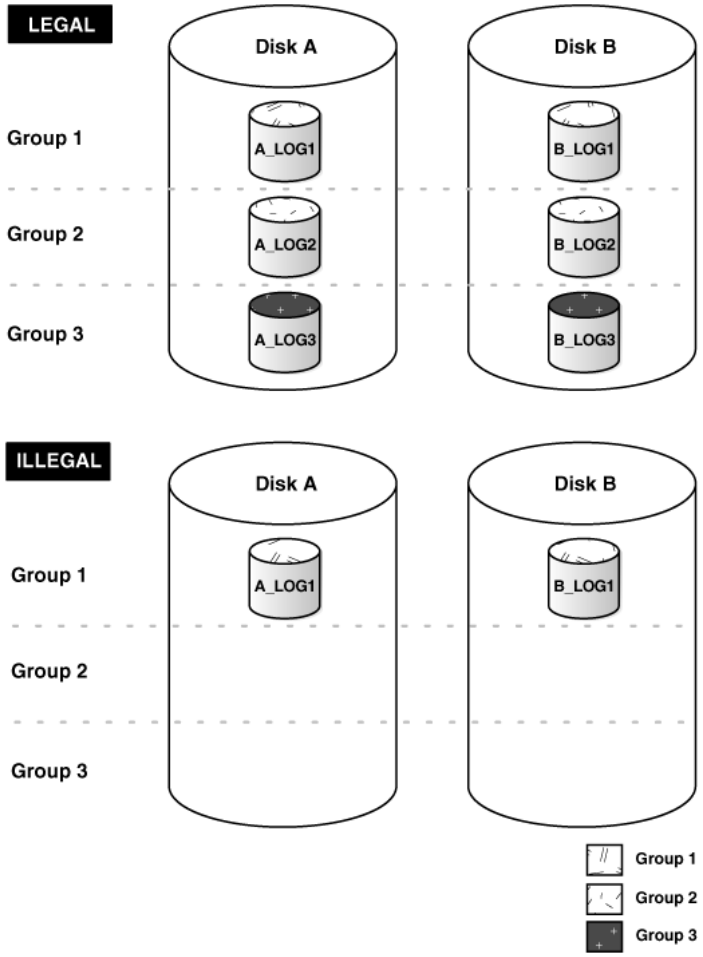
- Chaque groupe contient plusieurs membres
- Au moins deux groupes
- Pour chaque groupe on a au moins un seul membre
- Un membre = un fichier physique = un fichier redo log

- Lire un fichier d'archive ➔ log miner

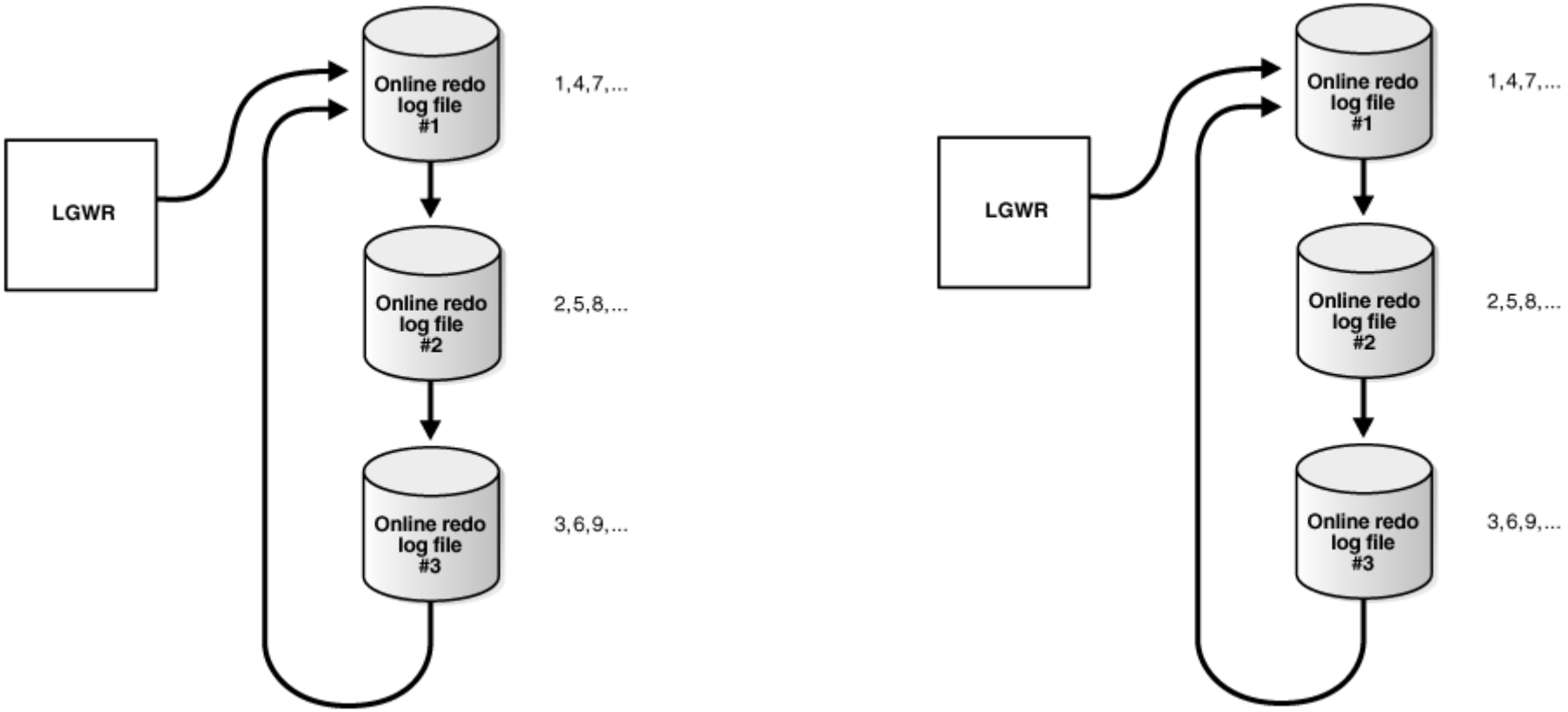
Groups –Redo log



Groups –Redo log



Groups –Redo log

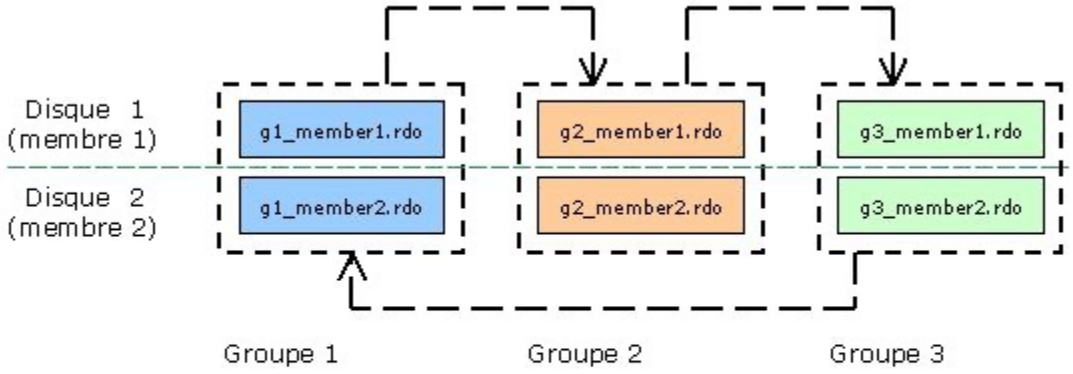


Multiplexage/séquentielle: à l'intérieur d'un groupe

Switch entre les groupes

- Ou bien basculement:
  - Le suitche entre les groupes se fait automatiquement si le groupe est plein
  - On peut faire manuellement via une commande sql
  - Raison su switch:
    - Si on veut supprimer un groupe
      - Switch, changer d'emplacement et puis le supprimer
    - ARC se déclenche automatiquement s'il y'a un groupe plein pour l'archiver

Switch entre les groupes



Ce schéma illustre une configuration :
- 3 groupes de Redo Logs
- Chaque membre d 'un groupe est multiplexé sur le disque 2
- Les flèches illustrent les basculements de redo logs (switch)

REDO LOG

- On garde le LOG pour 2 raisons:
  - ✓ Garder des traces pour ce voir ce qui a passé dans la BDD(modification effectuée sur les données)
  - ✓ Restauration /recouvrement(en cas de crash du système)

Donc pour la sécurité et la performance

- Utiliser LOGMINER pour lire le contenu des fichiers REDOLOG

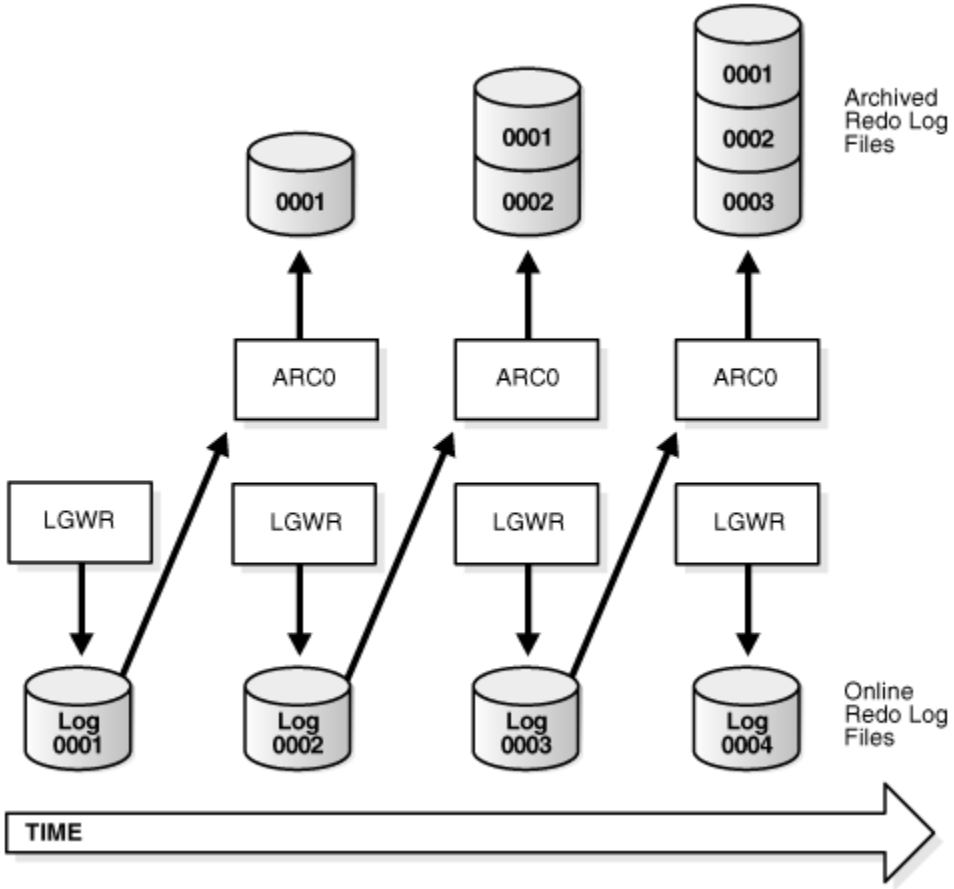
ARCHIVE (ARC*n*)

- Ecriture du contenu des fichiers journaux dans les fichiers journaux archivés.
- Déclenché par le basculement dans les fichiers journaux d'un groupe à un autre.
- Activé si le serveur fonctionne en mode `ARCHIVELOG`.
- Le paramètre `LOG_ARCHIVE_MAX_PROCESSES` spécifie le nombre d'exemplaires d'ARC*n*, même si le LGWR peut spécifier ce nombre de manière automatique (si pas d'intervention de l'administrateur).

ARC

- C’est le processus en arrière-plan responsable d’archivage
- C’est lui qui archive le membre d’un groupe si le groupe est plein (member = group)
- Il met le fichier archivé dans un autre composant dans le disque qui s’appelle archive

ARC


0001
Archived
Redo Log
Files
0001
0002
0001
0002
0003
ARC0
ARC0
ARC0
LGWR
LGWR
LGWR
LGWR
Log
0001
Log
0002
Log
0003
Log
0004
Online
Redo Log
Files
TIME

Streams pool

- Zone mémoire pour la gestion des flux
- C'est l'espace mémoire dans lequel Oracle streams travaille
- Espace pour la synchronisation et pas pour la restauration
- La synchronisation: en temps réel
- C'est le cache de données relatif à la queue de message utilisée par Oracle
- Une autre société a développé un autre produit pour la synchronisation qui s'appelle GOLDENGATE
  - Il se base sur oracle streams
  - Il est payant

Oracle goldenGate

Oracle GoldenGate is a software product that allows you to replicate, filter, and transform data from one database to another database.

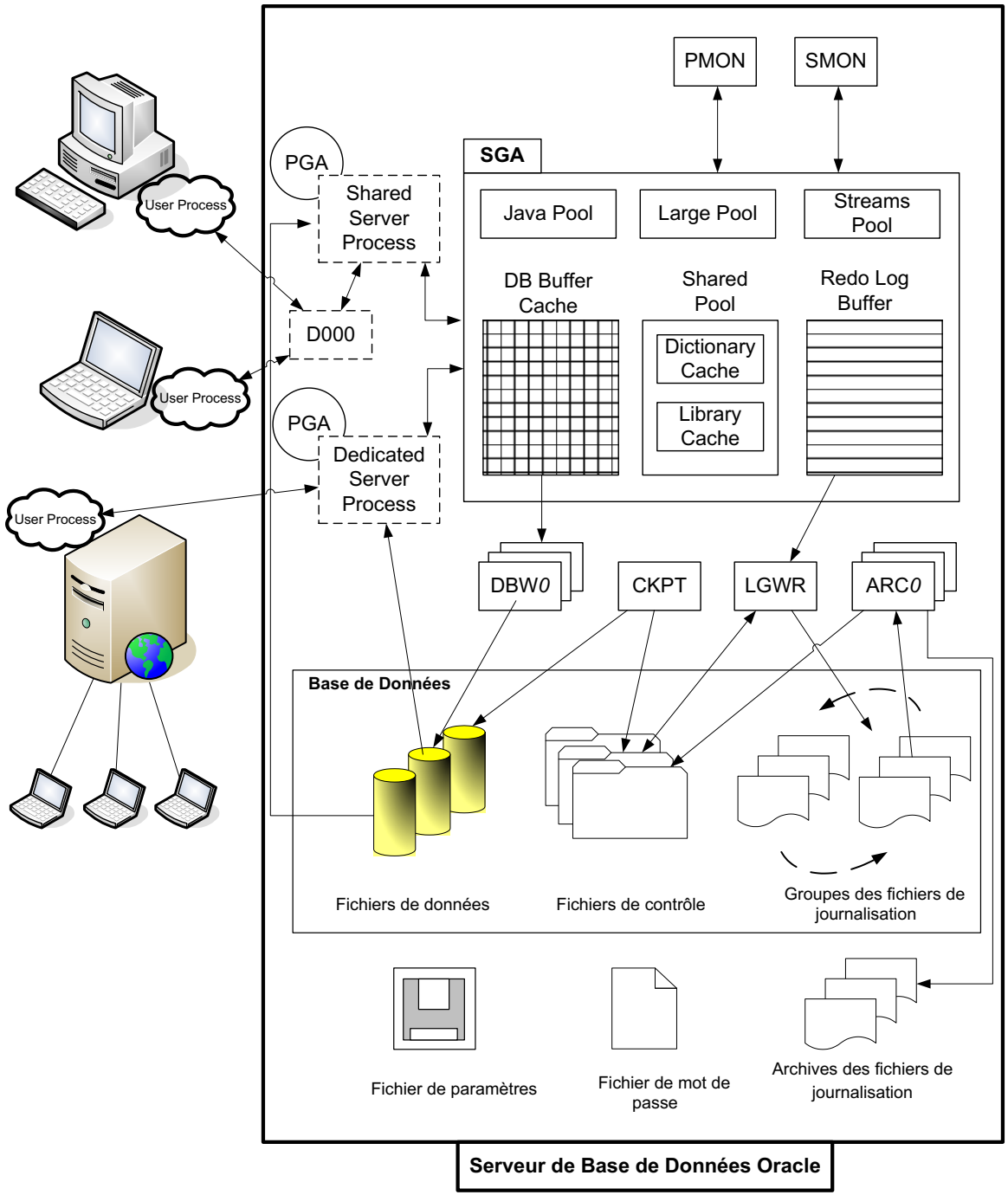
Using Oracle GoldenGate, you can move committed transactions across multiple heterogeneous systems in your enterprise. Oracle GoldenGate enables you to replicate data between Oracle databases to other supported heterogeneous database, and between heterogeneous databases. In addition, you can replicate to Java Messaging Queues, Flat Files, and to Big Data targets in combination with Oracle GoldenGate for Big Data.

Les processus d'arrière-plan

- Il existe d'autres processus d'arrière-plan, nous nous arrêterons sur l'ARC*n*.

- Le paramètre d'initialisation `PROCESSES` définit le nombre max de processus connectés en même temps à l'instance.

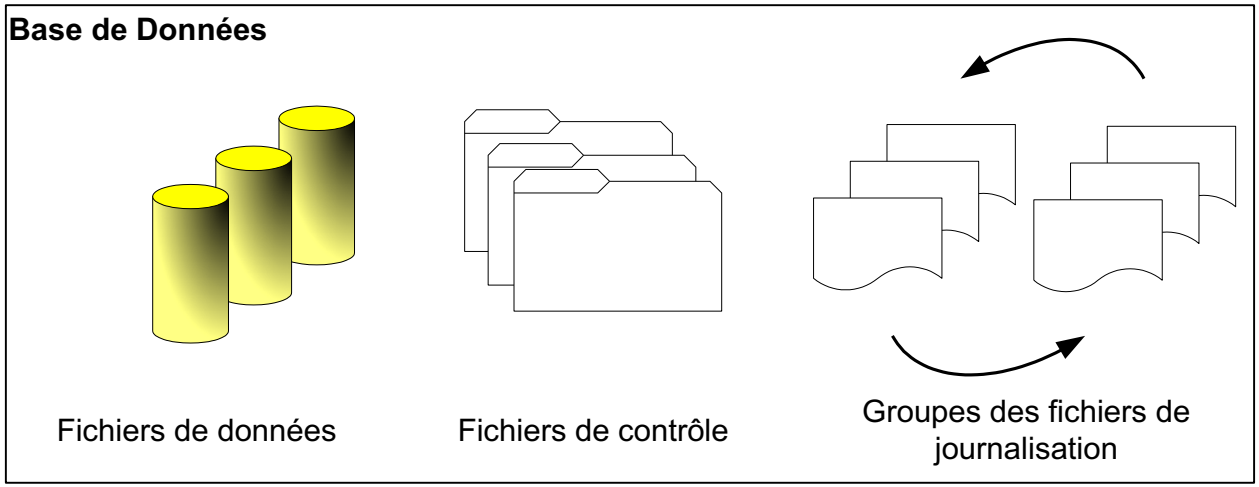
- La vue `V$PROCESS` inclut les informations de tous les processus en cours d'exécution dans l'instance.

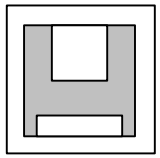

PMON
SMON
SGA
PGA
Shared Server Process
Java Pool
Large Pool
Streams Pool
DB Buffer Cache
Shared Pool
Redo Log Buffer
Dictionary Cache
Library Cache
User Process
D000
PGA
Dedicated Server Process
User Process
User Process
DBW0
CKPT
LGWR
ARC0
Base de Données
Fichiers de données
Fichiers de contrôle
Groupes des fichiers de journalisation
Fichier de paramètres
Fichier de mot de passe
Archives des fichiers de journalisation
Serveur de Base de Données Oracle

Le fast COMMIT & le SCN

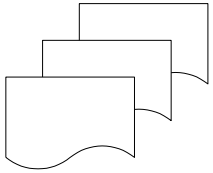
- La notion de fast COMMIT est liée à l'écriture différée des blocs dirty confirmés sur les fichiers de données.
- Dans le cas où après un **COMMIT**, seules les entrées **Redo** de la **transaction confirmée** sont **écrites** sur les fichiers de **journalisation** (les blocs dirty ne sont pas écrits sur les fichiers de données par le DBW*n*), cela s'appelle Fast COMMIT.
- Le SCN (System Change Number) est un code affecté à toute transaction confirmée par un COMMIT.
- Le SCN est inscrit sur les entrées redo de la transaction confirmée sur les fichiers de journalisation.
- Ce système de codage est fondamental pour la restauration des données en cas de crash du système.

Base de données

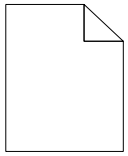




Fichier de paramètres

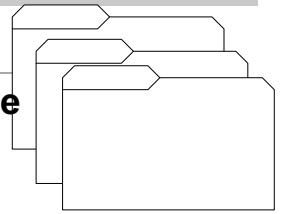


Archives des fichiers de journalisation



Fichier de mot de passe

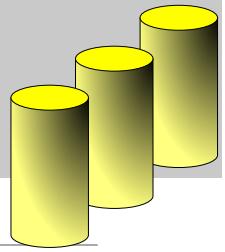
Fichier de contrôle



❑ Un fichier de contrôle (Control File)

- ❖ Contient des informations de contrôle sur la base: **Nom de la base**, **noms** et **chemins des fichiers de données** et de **journalisation**, **informations de restaurations** etc…
- ❖ *Un* des *premiers fichiers lus par l'instance* lors du *démarrage*.
- ❖ La vue `V$CONTROLFILE` nous renseigne sur le contenu du fichier de contrôle.

Fichier de données



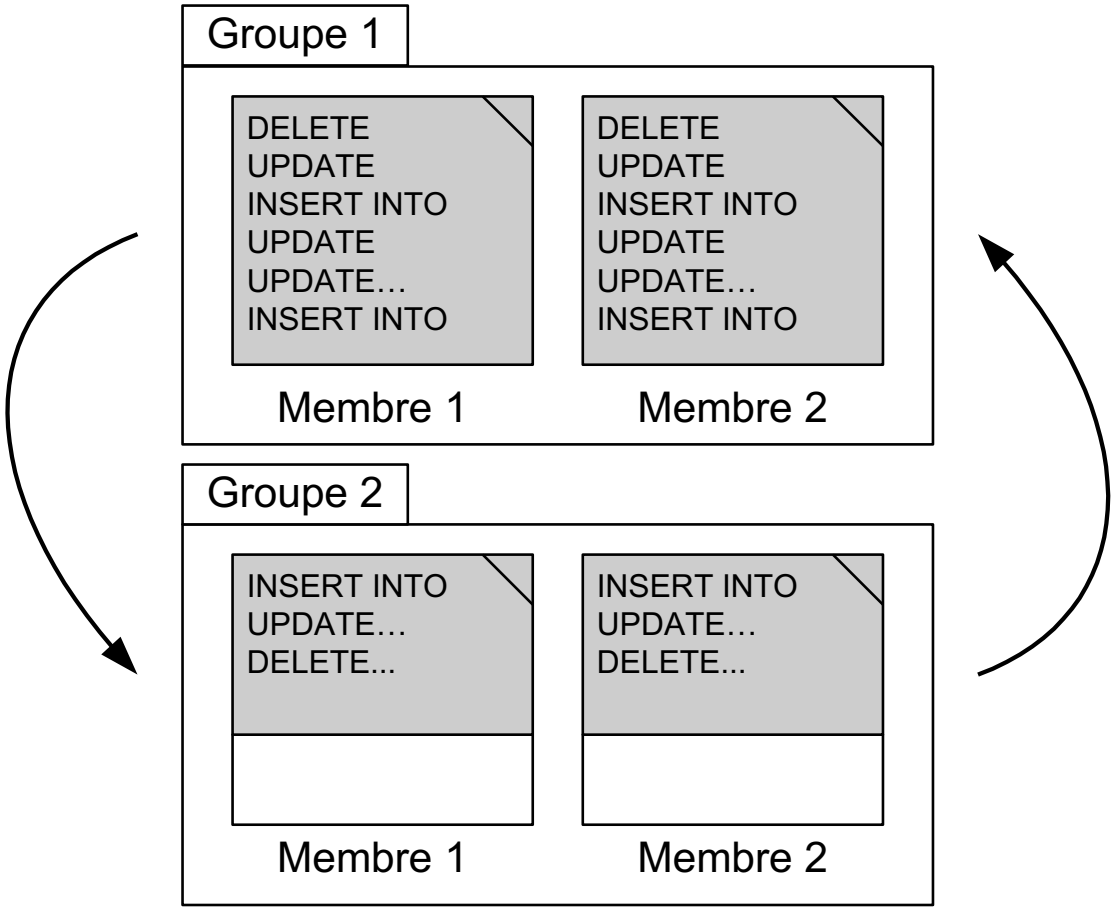
❑ Fichier de données (Data Files)

- ❖ **Stockent les données** sous un format spécial à Oracle.
- ❖ **Stocke le dictionnaire de données.**
- ❖ **Stocke les objets utilisateurs**.
- ❖ Stocke les **blocs de données** qui sont modifiés par les transactions courantes juste avant d'être modifiés
- ❖ Physiquement, un fichier de données est un ensemble de blocs SE. Un bloc SE constitue l'unité d'E/S (écriture/lecture) des fichiers.
- ❖ Les fichiers de données sont logiquement regroupés sous forme de **tablespaces**.
- ❖ Une BD Oracle inclut au moins deux tablespaces, **SYSTEM** et **SYSAUX**.

Fichiers de journalisation

- Les fichiers de journalisation contiennent toutes les **modifications** effectuées sur les données depuis une certaine durée.
- En cas de **crash du système**, ou d'altération des fichiers de données, on peut **reconstituer les données** à partir des fichiers journaux.
- L'écriture sur les fichiers journaux est **multiplexée** et **cyclique**.
- L'ensemble des fichiers multiplexés (qui contiennent donc les mêmes informations) sont appelés **membres** et forment un **groupe**.
- L'écriture est **multiplexée** à l'intérieur d'un groupe, et **cyclique** entre les groupes.
- La vue V$LOGFILE contient les informations des fichiers journaux.

Fichiers de journalisation



Archives des fichiers journaux

- L'écriture des fichiers journaux est cyclique, ce qui fait qu'à un certain moment, certaines transactions seront écrasées.
- La solution, c'est d'archiver les fichiers journaux avant de les écraser.
- Les fichiers journaux archivés peuvent être stockés sur des disques (serveurs) distants, ce qui optimisera la sécurité de la BD.
- Une base de données n'est pas forcément en mode ARCHIVELOG, si elle ne l'est pas, les fichiers journaux ne sont pas archivés.

Fichiers de REDO LOG

- Une base de données Oracle comporte au moins deux groupes de fichiers de journalisation, **chaque groupe comporte au moins un membre redo log**. Ces fichiers servent à enregistrer tous les changements effectués sur les données pour assurer leur restauration en cas de disfonctionnement du système.
- Pour plus de sécurité, oracle écrit dans les fichiers de journalisation d'une manière multiplexée ➔ sur plusieurs disques dur distants.

Autres fichiers

- Ce sont les fichiers qui ne font pas partie de la base de données mais qui sont utilisés par le serveur Oracle pour des opérations telles que le démarrage d'une instance ou l'authentification des utilisateurs.

- Il existe trois types de fichiers :
  - **Les fichiers de paramètre (Parameter File)** : Il définissent les *caractéristiques de l'instance*. Lors du démarrage d'une instance, le serveur Oracle va lire les *paramètres d'initialisation* du fichier de paramètres. On retrouve des informations telles que le *nom de la base de données*, la *taille de la mémoire à allouer*, le *nom des fichiers de contrôle* ainsi que les *autres* paramètres système.
  - **Le fichier de mots de passe (Password File)** : Il est utilisé pour authentifier les utilisateurs privilégiés.
  - **Les fichiers de redo log archivés (Archived Redo Log Files)** : Ils contiennent des copies *off-line* des fichiers de redo log. Ils sont utilisés lors d'une récupération suite à une défaillance d'un média. Les fichiers de redo log sont sauvés lorsqu'ils sont pleins, cette fonctionnalité n'est disponible que lorsque la base de données est en mode "ARCHIVELOG".

Les fichiers de paramètres

- Un fichier de paramètres inclut l'ensemble des paramètres de configuration du serveur BD.
- L'instance lit ce fichier et fonctionne selon les valeurs des paramètres qui y sont spécifiés.
- Il existe deux types de fichiers de paramètres:

PFILE (parameter file)	SPFILE (server parameter file)
Fichier texte	Fichier binaire
Modifiable via un éditeur texte	Modifiable via SQL
Disponible sur la machine de démarrage	Centralisé (sur le serveur uniquement)
Nommé init%.ora	Nommé spfile%.ora
Modification à froid	Modification à chaud

Les fichiers de paramètres

d

- Consultable via la vue V$PARAMETER
- SELECT name, value FROM V$PARAMETER

Le fichier mot de passe

❑ Oracle doit authentifier le DBA avant que la base ne soit ouverte, et donc le mot de passe DBA est enregistré hors de la BD dans un fichier de mot de passe

❑ L'authentification du DBA se fait via le ***système d'exploitation*** (`REMOTE_LOGIN_PASSWORDFILE=none`), ou bien via le ***fichier de mot de passe*** (`REMOTE_LOGIN_PASSWORDFILE=exclusive ou shared`).

- `none` : Authentification via le SE uniquement
- `exclusive` : utilisation d'un fichier de mot de passe dédié à une seule BD. Possibilité de donner le rôle `SYSDBA` ou `SYSOPER` à d'autres utilisateurs
- `shared` : utilisation d'un fichier de mot de passe partagé par plusieurs BDs. Seul SYS peut utiliser les privilèges `SYSDBA` ou `SYSOPER`

CHECKPOINT

- A checkpoint is a database event which synchronizes the modified data blocks in memory with the data files on disk.
- It also updates the datafile headers and control file with the latest checkpoint System Change Number (SCN).
- We need checkpoint to:
  - Establish data consistency
  - Enable faster database recovery

SCN (System change number)

Instance recovery

- Roll Forward
- Roll back

Backup and Recovery

Read Consistency

- A system change number (SCN) is an internal timestamp for a committed version of the database.
- The Oracle Database server uses the SCN clock value to guarantee transaction consistency.
- For example, when a user commits a transaction, the database records an SCN for this commit in the redo log.
- The server uses SCNs to coordinate distributed transactions among different databases.
- For example, the server uses SCNs in the following way:
  1. An application establishes a connection using a database link.
  2. The distributed transaction commits with the highest global SCN among all the databases involved.
  3. The commit global SCN is sent to all databases involved in the transaction.

select

DML

DLL

COMMIT

When will a checkoint occur?

1) at each switch of the redo log files.
2) once the number of seconds defined in the LOG_CHECKPOINT_TIMEOUT is reached.
3) once the current redo log file reaches the size:

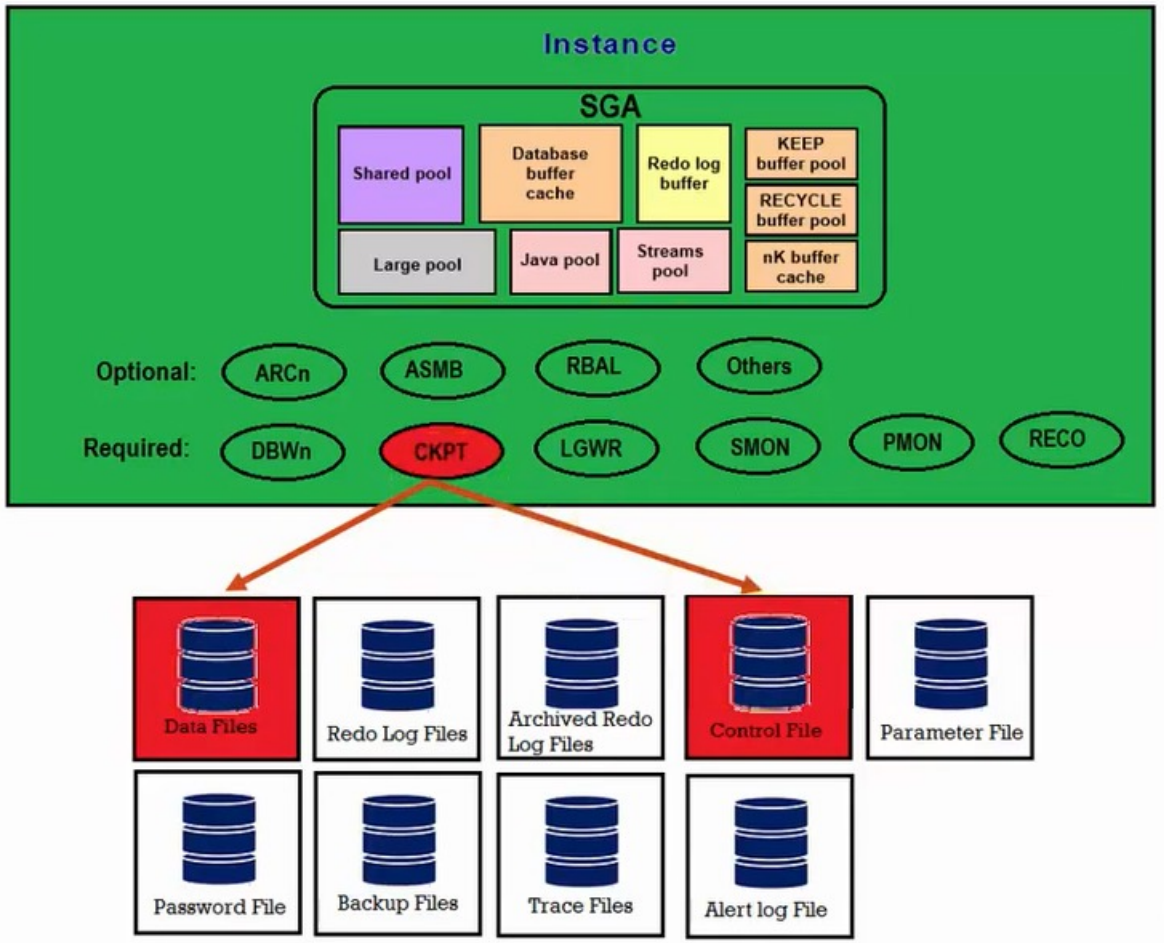
LOG_CHECKPOINT_INTERVAL*size of IO OS blocks

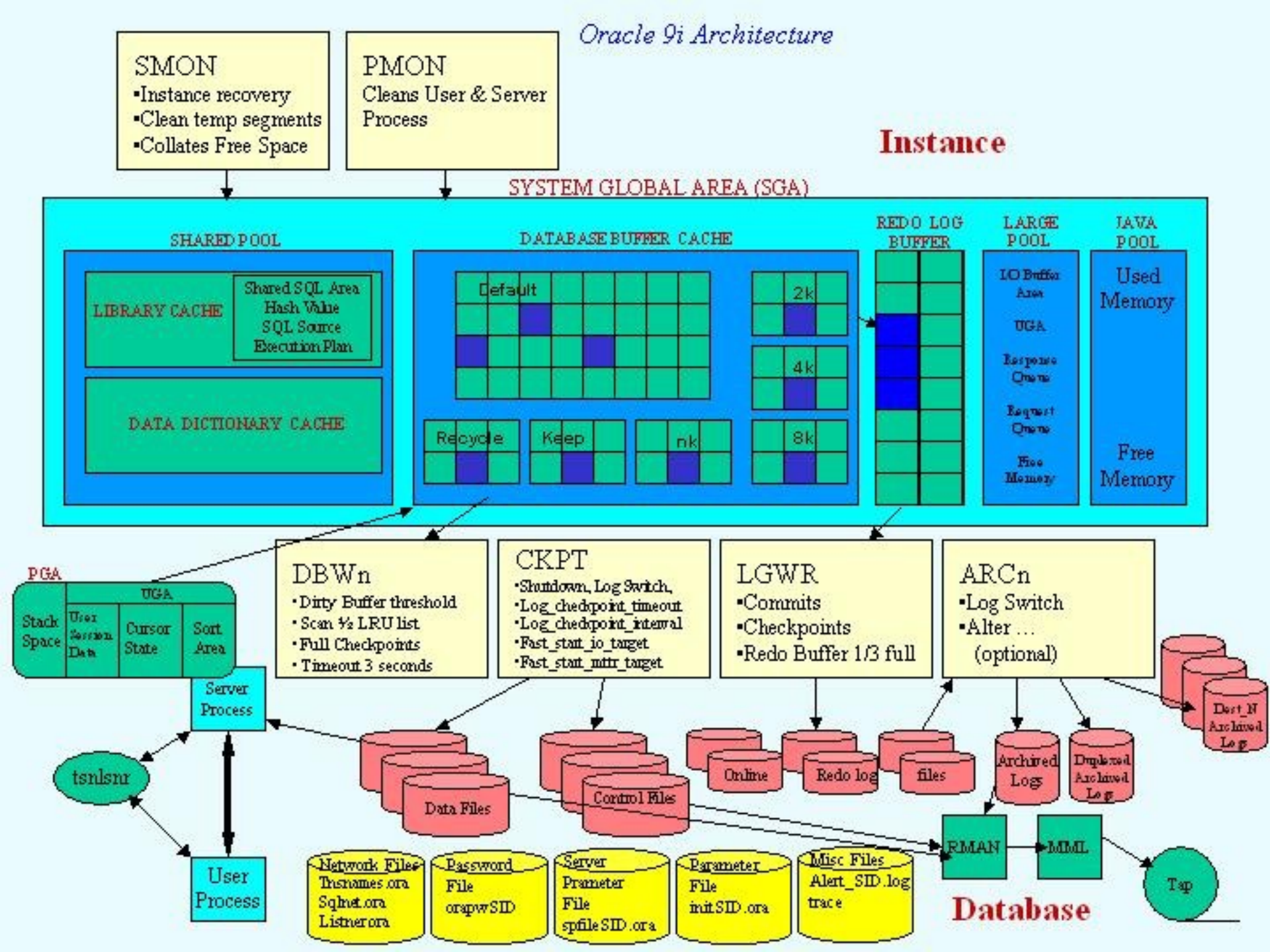
Example: 10000*512= 5.120.000 (5M)

4) Directly by the ALTER SYSTEM SWITCH LOGFILE command.
5) directly with the ALTER SYSTEM CHECKPOINT command.

CHECKPOINT

Recors checkpoint Information (SCN) into the control file and each data file header.




Oracle 9i Architecture
SMON
•Instance recovery
•Clean temp segments
•Collates Free Space
PMON
Cleans User & Server Process
Instance
SYSTEM GLOBAL AREA (SGA)
SHARED POOL
LIBRARY CACHE
Shared SQL Area
Hash Value
SQL Source
Execution Plan
DATA DICTIONARY CACHE
DATABASE BUFFER CACHE
Default
2k
4k
8k
Recycle
Keep
nk
REDO LOG BUFFER
LARGE POOL
IO Buffer Area
UGA
Response Queue
Request Queue
Free Memory
JAVA POOL
Used Memory
Free Memory
PGA
Stack Space
UGA
User Session Data
Cursor State
Sort Area
Server Process
User Process
tnslsnr
DBWn
• Dirty Buffer threshold
• Scan ½ LRU list
• Full Checkpoints
• Timeout 3 seconds
CKPT
•Shutdown, Log Switch,
•Log_checkpoint_timeout
•Log_checkpoint_interval
•Fast_start_io_target
•Fast_start_mttr_target
LGWR
•Commits
•Checkpoints
•Redo Buffer 1/3 full
ARCn
•Log Switch
•Alter …
(optional)
Dest_N Archived Log
Data Files
Control Files
Online
Redo log
files
Archived Logs
Duplexed Archived Log
RMAN
MML
Tap
Network_Files
Tnsnames.ora
Sqlnet.ora
Listner.ora
Password File
orapwSID
Server Parameter File
spfileSID.ora
Parameter File
initSID.ora
Misc Files
Alert_SID.log
trace
Database

Asynchronous I/O

- Synch I/O = Blocking I/O where a process waits (does not return) till the OS completes the I/O
- Asynch I/O = Non-Blocking I/O where a process does not wait for the OS to complete the I/O, but rather continues with its processing and the OS sends back a signal to the process to indicate the I/O completion.
- DISK_ASYNCH_IO controls whether I/O to datafiles, control files, and logfiles is asynchronous (that is, whether parallel server processes can overlap I/O requests with CPU processing during table scans). If your platform supports asynchronous I/O to disk, Oracle recommends that you leave this parameter set to its default value. However, if the asynchronous I/O implementation is not stable, you can set this parameter to false to disable asynchronous I/O. If your platform does not support asynchronous I/O to disk, this parameter has no effect.

If you set DISK_ASYNCH_IO to false, then you can increase DB_WRITER_PROCESSES or use DBWR_IO_SLAVES to simulate asynchronous I/O.

DB_WRITER_PROCESSES

- DB_WRITER_PROCESSES specifies the initial number of Database Writer Processes for an instance. This parameter is useful for systems that modify data heavily.
- Default value: 1 or CPU_COUNT / 8, whichever is greater. If the number of processor groups is less than 100 but greater than the number of Database Writer Processes, then the number of Database Writer Processes is adjusted to be a multiple of the number of processor groups. If the number of Database Writer Processes is greater than or equal to the number of processor groups, then there is no adjustment.
- Range of values: 1 to 100
  - There can be 1 to 100 Database Writer Processes. The names of the first 36 Database Writer Processes are DBW0-DBW9 and DBWa-DBWz. The names of the 37th through 100th Database Writer Processes are BW36-BW99.

DBWR_IO_SLAVES

If it is not practical to use multiple DBWR processes, then Oracle Database provides a facility whereby the I/O load can be distributed over multiple slave processes. The DBWR process is the only process that scans the buffer cache LRU list for blocks to be written out. However, the I/O for those blocks is performed by the I/O slaves. The number of I/O slaves is determined by the parameter DBWR_IO_SLAVES.

DBWR_IO_SLAVES

DBWR_IO_SLAVES is intended for scenarios where you cannot use multiple DB_WRITER_PROCESSES (for example, where you have a single CPU). I/O slaves are also useful when asynchronous I/O is not available, because the multiple I/O slaves simulate nonblocking, asynchronous requests by freeing DBWR to continue identifying blocks in the cache to be written. Asynchronous I/O at the operating system level, if you have it, is generally preferred. DBWR I/O slaves are allocated immediately following database open when the first I/O request is made. The DBWR continues to perform all of the DBWR-related work, apart from performing I/O. I/O slaves simply perform the I/O on behalf of DBWR. The writing of the batch is parallelized between the I/O slaves.

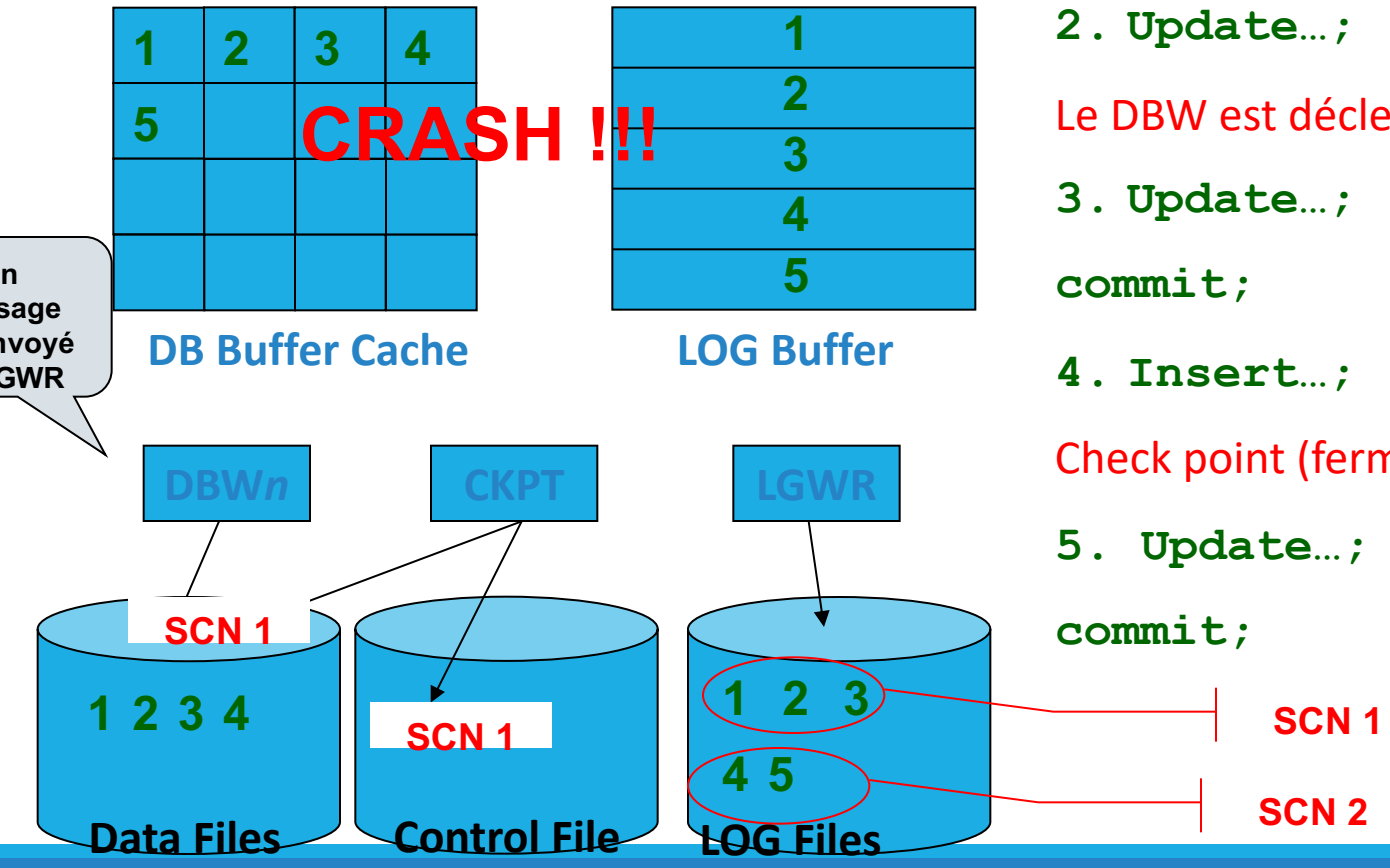
BACKUP_TAPE_IO_SLAVES

BACKUP_TAPE_IO_SLAVES specifies whether I/O server processes (also called slaves) are used by Recovery Manager to back up, copy, or restore data to tape. When the value is set to true, Oracle uses an I/O server process to write to or read from a tape device. When the value is set to false (the default), Oracle does not use I/O server process for backups. Instead, the shadow process engaged in the backup accesses the tape device.

Le Check Point (CKPT)

- Le CKPT est lancé lorsque le système a besoin de faire un check point (point de vérification).
- Lorsqu'un point de vérification est déclenché, le CKPT envoie un message au DBW*n* pour qu'il écrive tous les blocs dirty sur les fichiers de données.
- Une fois le DBW*n* termine sa tâche, le CKPT écrit le SCN de la dernière transaction confirmée dans les entêtes des fichiers de données ainsi que dans le fichier de contrôle.
- En cas de crash, les transactions récupérées à partir des fichiers journaux sont celles dont le SCN est supérieur à celui écrit dans le fichier de contrôle.

Exemple




```
1. Insert…;
2. Update…;
```

Le DBW est déclenché !

```
3. Update…;
commit;
4. Insert…;
```

Check point (fermeture du tblsp) !

```
5. Update…;
commit;
```

Les tablespaces

- Un *tablespace* est une unité logique qui correspond physiquement à un ou plusieurs fichiers de données.
- L'administrateur (DBA) agit sur les tablespaces et non sur les fichiers de données.
- Une BD est organisée sous forme de plusieurs tablespaces, chacun correspondant à un contexte (thème), pour bien ranger les objets par type (applications, index, table, …) et ainsi permettre une administration plus fluide et plus souple, une meilleure optmisation des performances.
- EX: On peut créer plusieurs tablespaces dans une BD d'une Entreprise commerciale qui gère la FACTURATION, la GRH, et le PARC INFO.

Les tablespaces

- Le tablespace FACTURATION → { fact01.dbf et fact02.dbf }
  - Les tables FACTURE, LIGNE_FACTURE, PRODUIT, CLIENT, REGLEMENT.
- Le tablespace GRH → { grh01.dbf }
  - Les tables PERSONNEL, PAIE, SANCTION etc.
- Le tablespace PARC → { parc01.dbf }
  - Les tables EQUIPEMENT, CATEGORIE, REPARATION, MAINTENANCE etc.
- Avantage: On peut administrer par tablespace, et donc par partie (par rapport à la BD). Par exemple, pour maintenir certaines tables relatives à la facturation, on peut mettre uniquement le tablespace FACTURATION en offline, au lieu de rendre toute la base indisponible ce qui touchera des centaines d'utilisateurs qui ont besoin de manipuler les données de la GRH, PARC INFO etc.

Les tablespaces

- Les vues **DBA_TABLESPACES** et **DBA_DATA_FILES** incluent toutes les informations relatives aux tablespaces et aux fichiers de données de la base.

- Pour afficher les noms des fichiers de données ainsi que les tablespaces auxquelles ils correspondent:

```
SELECT tablespace_name, file_name

FROM DBA_DATA_FILES

ORDER BY tablespace_name;
```

Les tablespaces

- Une base de données Oracle possède au minimum 4 tablespace:
  - Tablespace SYSTEM dictionnaire de données
  - Tablespace SYSAUX system auxiliaire composants oracle
  - Tablespace UNDO segments d'annulations
  - Tablespace TEMPORAIRE segments temporaires
- Puis des tablespace spécifiquement créés pour des données applicatives et utilisateurs

Le tablespace UNDO

- Le tablespace UNDO, comme son nom l'indique, est réservé exclusivement à l'annulation des commandes DML (Data Manipulate Language) (UPDATE, INSERT, etc...).
- Lorsqu'on exécute l'ordre DELETE par exemple, Oracle commence par copier les lignes à supprimer dans le tablespace UNDO et ensuite indique que les blocs contenant les données dans le tablespace d'origine sont libres.
- Un ROLLBACK permet de revenir en arrière alors que le COMMIT supprimera les lignes du tablespace UNDO.

Création d'un tablespace

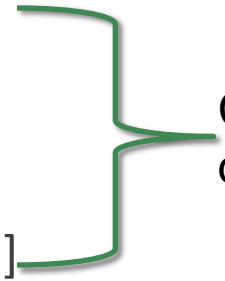
CREATE TABLESPACE nom_tablespace
DATAFILE 'chemin d'accès du fichier' **SIZE** valeur [K|M]
[MINIMUM EXTENT valeur [K|M]]
[MAXIMUM EXTENT valeur [K|M]]
[DEFAULT STORAGE
([INITIAL valeur [K|M]]
[NEXT valeur [K|M]]
[MINEXTENTS nombre]
[MAXEXTENTS nombre]
[PCTINCREASE pourcentage]) ;

DATAFILE: chemin d'accès du fichier dans lequel est stocké le tablespace
Ex : 'C:\oracle\oradata\data_tbs.dbf'

SIZE: Taille du fichier (tablespace)

Création d'un tablespace

- MINIMUM EXTENT: Taille minimale d'une extension
- MAXIMUM EXTENT : Taille maximale d'une extension
- ([INITIAL valeur [K|M]]
  [NEXT valeur [K|M]]
  [MINEXTENTS nombre]
  [MAXEXTENTS nombre]
  [PCTINCREASE pourcentage]

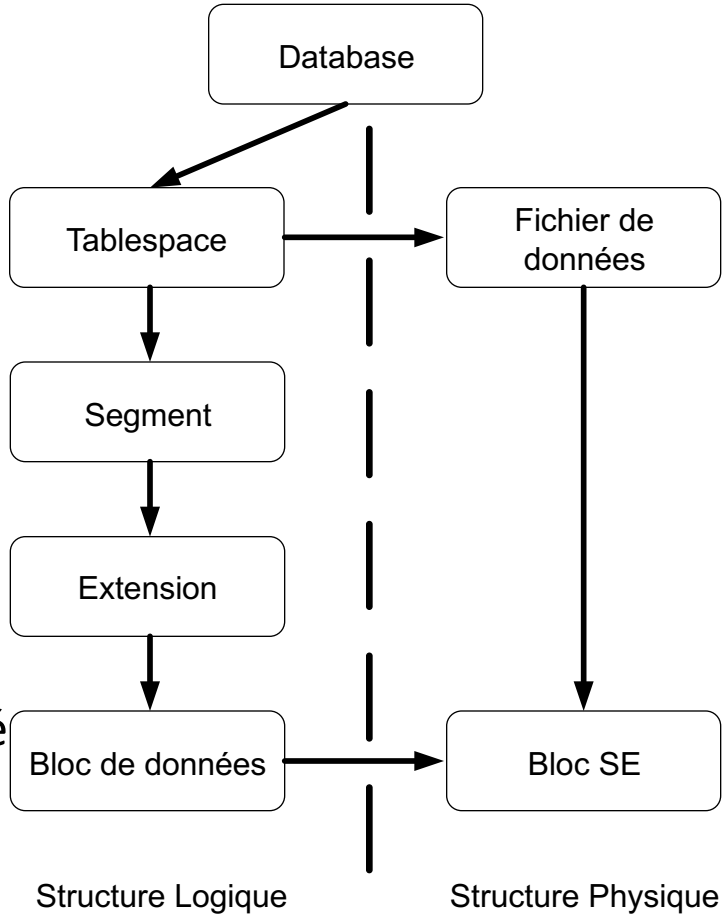


Caractéristiques du stockage

  - INITIAL : Taille de la première extension allouée lors de la création d'un segment
  - NEXT : Taille de la deuxième extension du segment
  - MINEXTENTS : Nombre d'extensions allouées à la création du segment
  - MAXEXTENTS : Nombre maximal d'extensions pouvant être allouées au segment
  - PCTINCREASE : Pourcentage d'accroissement de la taille des extensions, appliqué à partir de la troisième extension

Structure de stockage

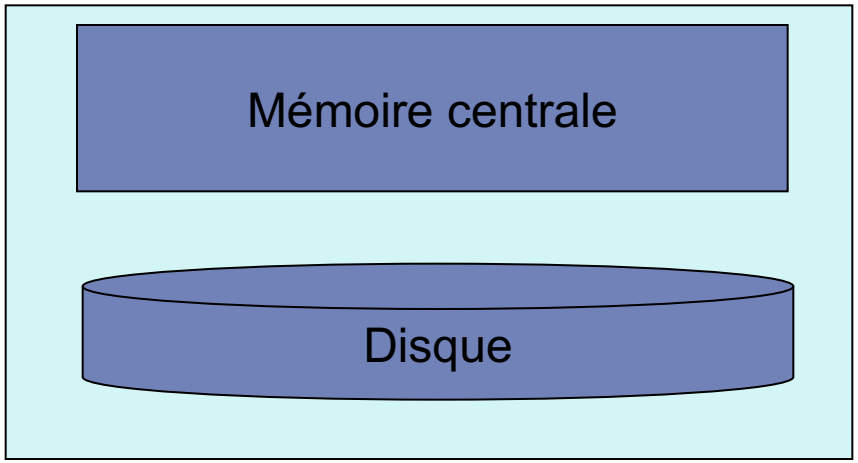
- C'est l'espace occupé par un objet base de données (Table ou Index)
- Il existe 4 types de segments:
  1. **Segment table**: espace occupé par les tables
  2. **Segment index**: espace occupé par un index
  3. **Segment d'annulation**: espace temporaire pour stocker les informations pour annuler une transaction
  4. **Segment temporaire**: espace temporaire utilisé pour les opérations volumineuses. Le tri …



Le segment d'annulation

- **Le segment d'annulation** sert à stocker les données nécessaires:

1. A l'annulation

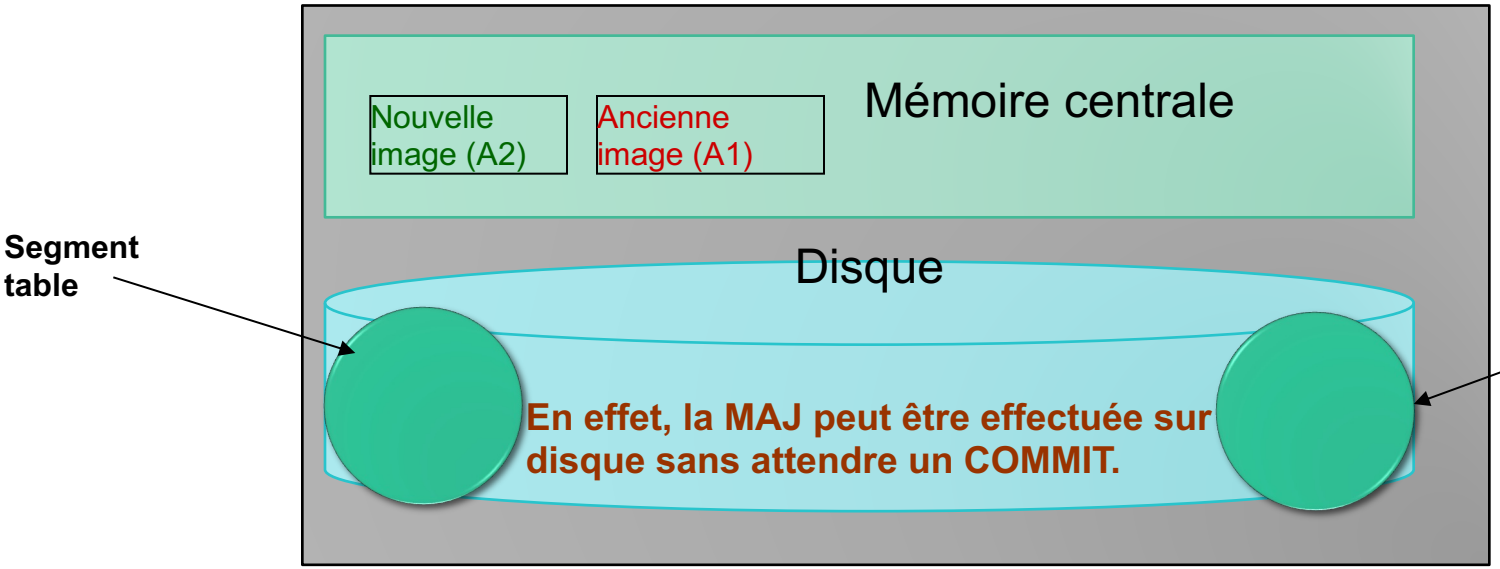



```
Update table1
set col1=A2
Where col1=A1;
```

Le segment d'annulation

❑ **Le segment d'annulation** sert à stocker les données nécessaires:

1. A l'annulation



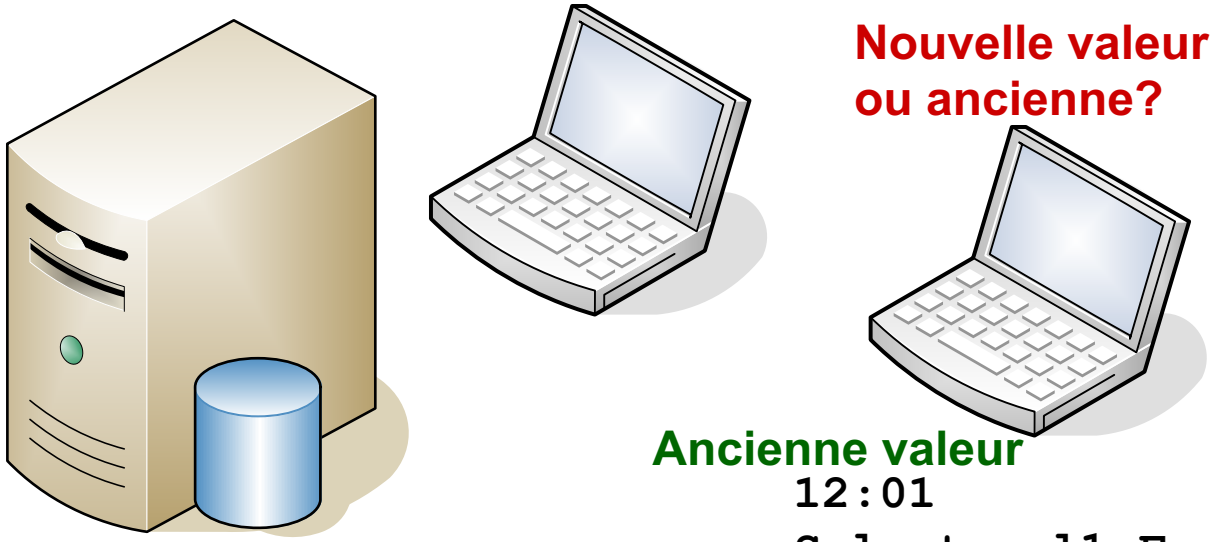

```
Update table1
Set col1=A2
Where
col1=A1;
```

La nouvelle image est enregistrée dans un segment table, l'ancienne dans un segment d'annulation. Au cas où un ROLLBACK est effectué, c'est cette image qu'on utilisera pour rétablir les données.

Le segment d'annulation

- **Le segment d'annulation** sert à stocker les données nécessaires:

2. À la lecture cohérente (ou dite aussi consistante).



Le segment d'annulation

- **Le segment d'annulation** sert à stocker les données nécessaires :

1. À l'annulation
2. À la lecture cohérente (ou dite aussi consistante).
   a. Si pas d'anticipation d'enregistrement des MAJ, Alors la lecture se fait directement à partir du segment table.
   b. Si anticipation d'enregistrement des MAJ, Alors la lecture se fait à partir du segment d'annulation.

Le segment d'annulation

- Read consistency
- Statement read consistency
- Transaction read consistency

➔ Statement read consistency:

Select starts 10am on 12 Nov 22

Ends at 11am on 12th Nov 22

All rows as it was at 10 am on 12th Nov 22







During this 1 hour period, other users could have performed DMLs on the same table

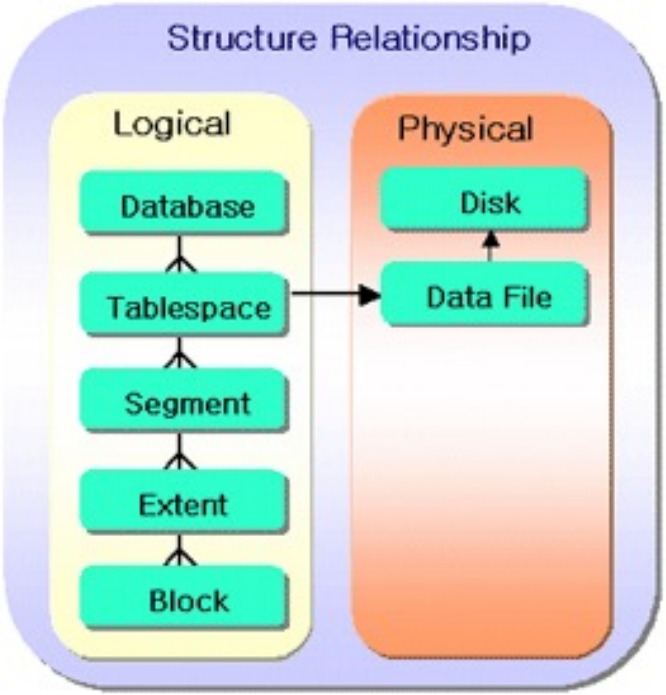
Le segment temporaire

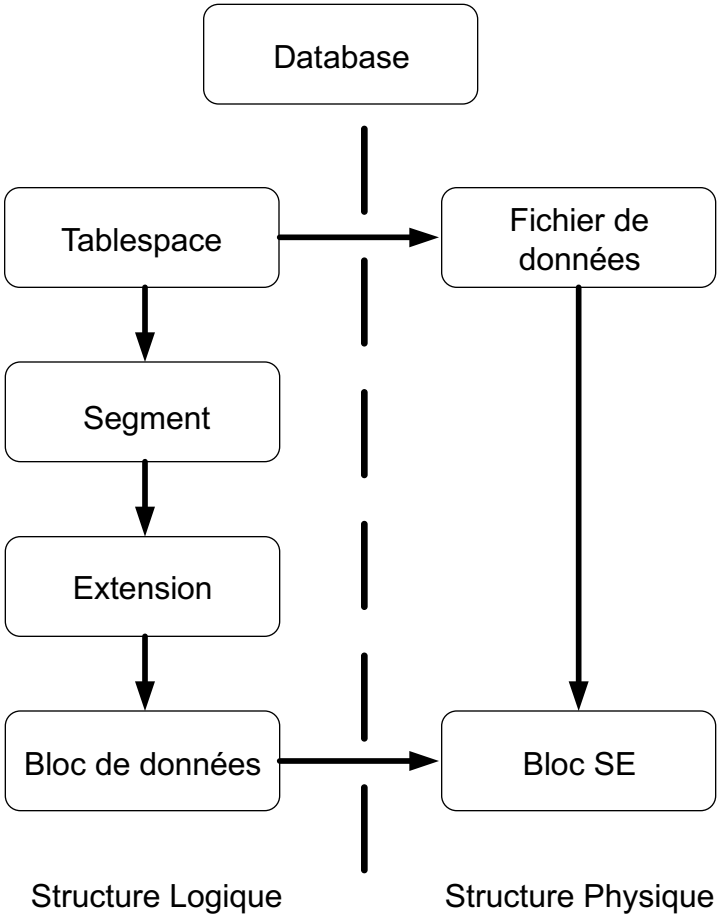
- **Le segment temporaire** sert à stocker les données relatives à des opérations volumineuses si la mémoire centrale ne suffit pas à les exécuter.

  Exemple d'opérations volumineuses:

  - Certains tris.
  - Certaines jointures.
  - Création d'index.
  - Etc.

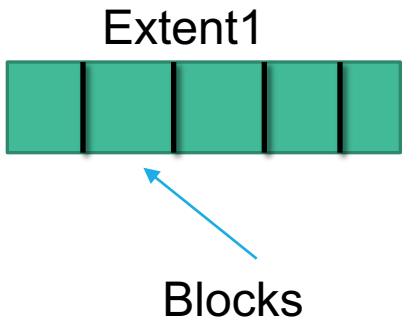
Logique vs Physique

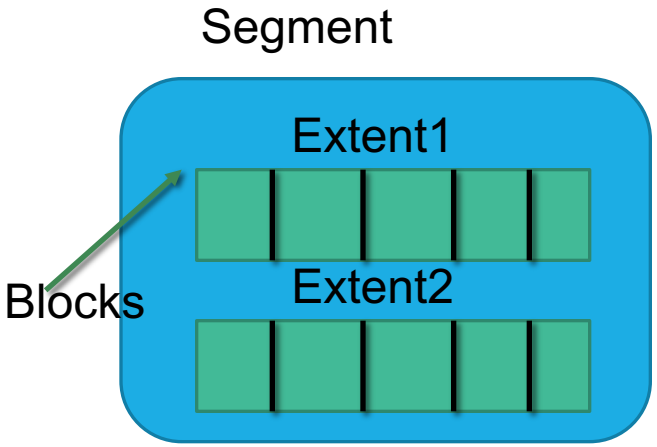




Extension

- Un **segment** est à son tour composé **d'extensions**
  - **Représente un objet de BDD comme table, index …**
- Une **extension** est un ensemble de **blocs** appartenant à un même fichier de données
- Par contre un segment peut s'étaler sur plusieurs fichiers à travers ses extensions.
- La taille d'un bloc de données est définie par le paramètre DB_BLOCK_SIZE



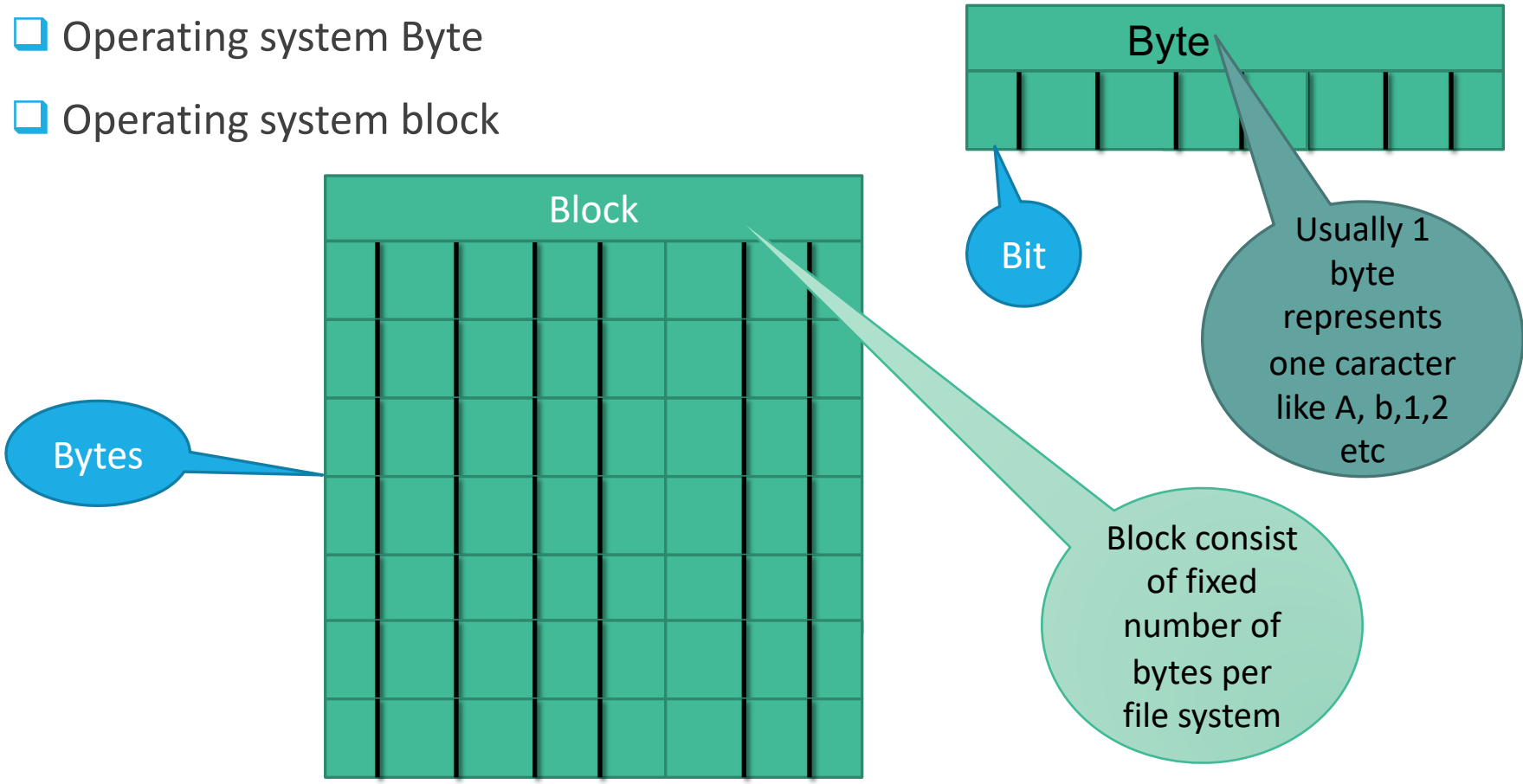


Segments

- A segment is created when you create a database object
- A segment is the smallest unit of storage that end-user will deal with
- When a database object, like table, is created a segment is created in the database
- Initially, fix number of extents are given to that segment
- The initial number of extents given to a segment, can be controlled
- The size of initial extents (number of blocks) could be controlled as well
- When the segment gets full, more extents are added to the segment
- The number of extents that are added can be controlled
- The size of the new extents could be controlled as well
- The segment keeps on growing as new data comes in

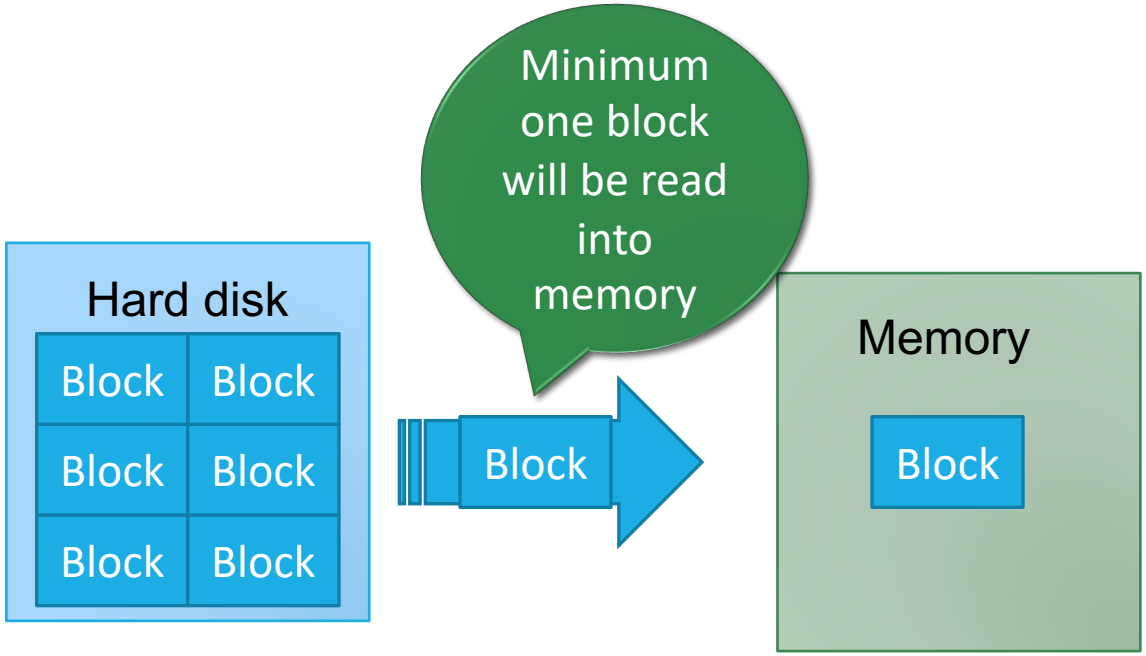
Blocks

- Operating system Byte
- Operating system block



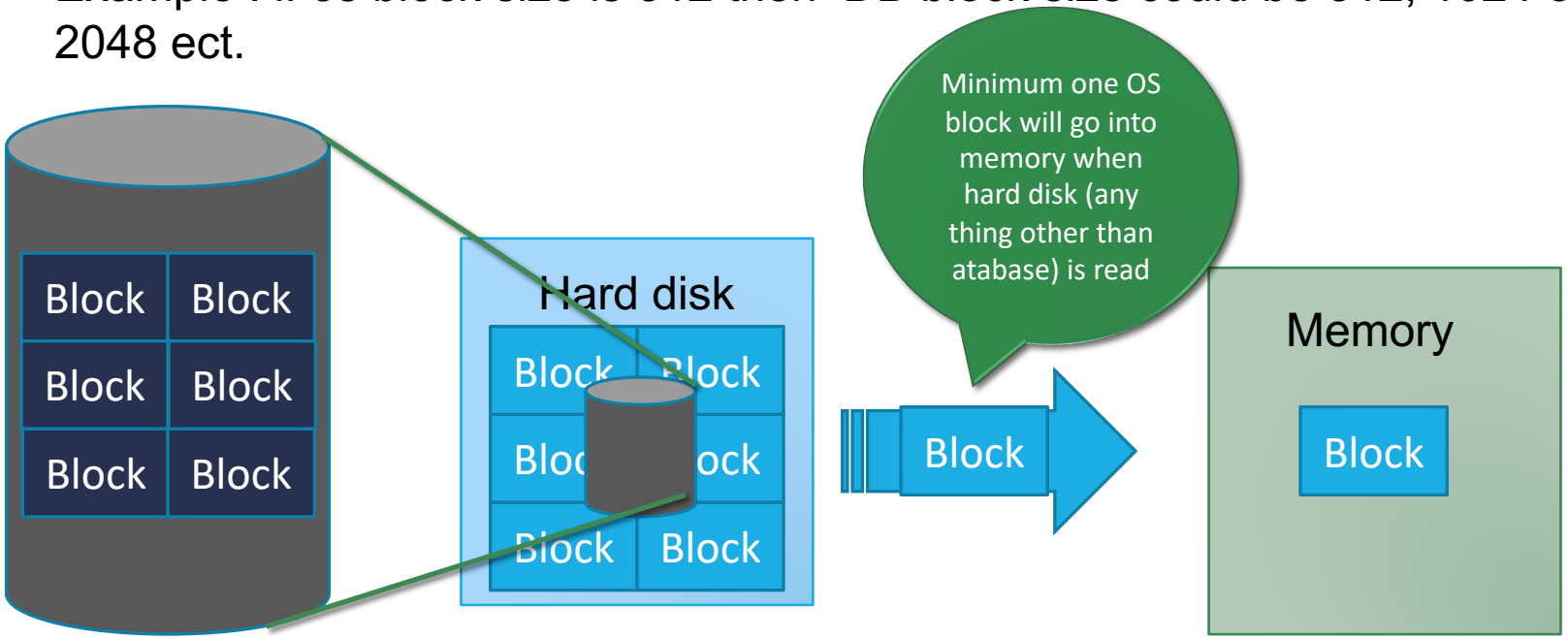
Blocks

A block is the smallest unit that will be read into the memory(CM)



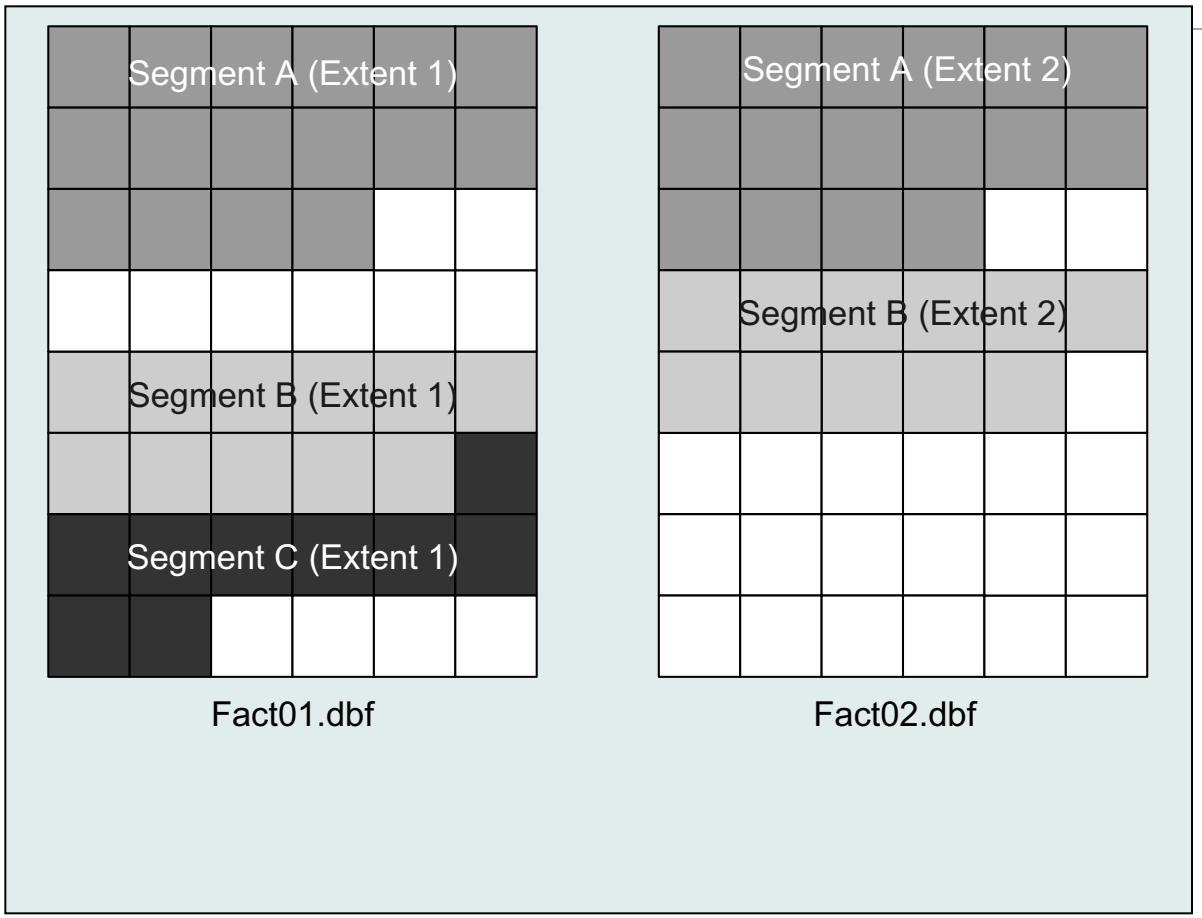
Database

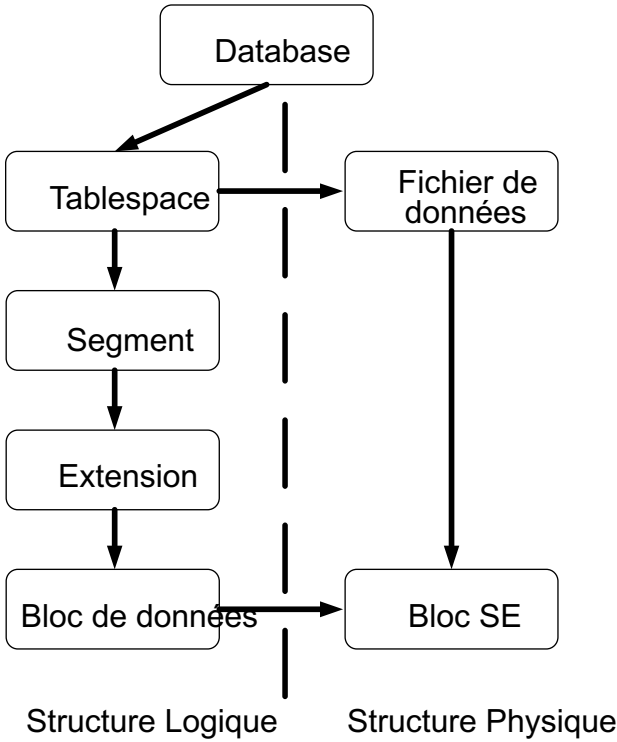
Db block size have to be multiple of OS block size
Example : if os block size is 512 then DB block size could be 512, 1024 or 2048 ect.



When even an OS block size will go into the memory when hard disk is read, but when DB is read which is nothing but a program on the hard disk but since DB instance running in the memory will read the DB, when these is read minimum one DB block will travel from hard disk into the memory. So one DB block would be one equal to one OS block or could be equal to two OS block or could be equal to 3 OS block

Exemple





Structure du tablespace FACTURATION

Schéma

- Ensemble d'objets qui ont été créés par un utilisateur.
- Chaque utilisateur est le propriétaire d'un unique schéma.
- Les principaux types d'objets d'un schéma sont les tables, les index, les vues, les synonymes, les séquences, les déclencheurs, les fonctions et procédures stockées et les packages PL/SQL.
- Seuls les tables et les index sont stockés sous forme de segments, *les autres objets correspondent à des définitions dans le dictionnaire de données.*
- Les objets d'un schéma peuvent être stockés sur différents tablespaces tandis qu'un unique tablespace peut inclure plusieurs schémas.

Rappel : Dictionnaire de données

- Ensemble de tables/vues accédées en lecture seule, créées et maintenues par le système.
- Elles contiennent toutes les informations de toutes les composantes logiques et physiques de la ***base de données*** ainsi que de ***l'instance***.
- Il est créé dans le tablespace SYSTEM, et c'est l'utilisateur SYS qui en est le propriétaire.
- Les tables contiennent des données codées. Pour y accéder, on consulte les vues.
- Il existe deux types de vues; les vues statiques et les vues dynamiques.

Dictionnaire de données – Les vues statiques

- Elles sont basées sur des **tables** créés réellement dans le dictionnaire de données.
- Accessibles **uniquement si la base est ouverte**. Commencent par les préfixes:
  - `USER_XXX` décrit les objets appartenant à l'utilisateur connecté.
  - `ALL_XXX` décrit les objets accessibles à l'utilisateur connecté.
  - `DBA_XXX` décrit tous les objets (vues autorisées aux DBAs seulement...)
    - select * from DICT where table_name= 'DBA_USERS'
- La vue `DICTIONARY` inclut des informations sur les vues statiques et dynamiques du dictionnaire de données.

Dictionnaire de données – Les vues statiques

Nom de la vue	Description
%_TABLES	Toutes les informations des tables de la base de données.
%_USERS	Toutes les informations concernant les utilisateurs de la base de données.
%_VIEWS	Toutes les informations des vues de la base de données.
%_SEQUENCES	Toutes les informations concernant les séquences de la base de données.
%_TAB_COLUMNS	Toutes les informations concernant les colonnes des tables de la base de données.
%_INDEXES	Toutes les informations concernant les index de la base de données.
%_OBJECTS	Toutes les informations des objets –tous types confondus- de la base de données.

Dictionnaire de données – Les vues dynamiques

- Elles ne sont **pas basées sur des tables** du dictionnaire de données. Leurs informations sont **extraites de la mémoire** et/ou **des fichiers de contrôle**.
- Appartiennent (comme les autres) a **SYS**
- Commencent par le préfixe ***V$*** et ne sont accessibles que par les **administrateurs**.
- Peuvent être ***consultées*** même si la ***base de données n'est pas ouverte***

Dictionnaire de données – Les vues dynamiques

Nom de la vue	Description
V$VERSION	Nom du composant logiciel et no de version du noyau Oracle
V$DATABASE	Informations de la base de données à partir du fichier de contrôle.
V$INSTANCE	Informations sur l'instance.
V$SGA	Informations résumées sur la SGA.
V$SGA_DYNAMIC_COMPONENTS	Informations détaillées sur les zones mémoire de la SGA.
V$PARAMETER	Information sur les différents paramètres de l'instance et de la BD.
V$OPTION	Informations des composantes optionnelles installées sur le serveur BD.
V$SQL	Informations des requêtes SQL exécutées par tous les utilisateurs de la BD.

Fonctionnement d'Oracle – Requête SELECT (1)

- Le serveur BD reçoit une requête SELECT

I. Phase de parse : le processus serveur vérifie si la requête existe dans le Library Cache.

   a. Si elle n'y existe pas (**Hard parse**)

      i. Une vérification syntaxique est faite

      ii. Vérification sémantique (exactitude des tables & colonnes utilisées, droits d'accès)

         1. Si le DC contient les informations sémantiques, pas d'accès au disque
         2. Sinon, accès au disque (dictionnaire de données)

      iii. Calcul d'un plan d'exécution

   b. Si elle existe (**Soft parse**)

      i. Seule la vérification des privilèges est effectuée

Fonctionnement d'Oracle – Requête SELECT (2)

- Le serveur BD reçoit une requête SELECT
  - I. Phase de parse : le processus serveur vérifie si la requête existe dans le Library Cache.
  - II. Phase d'exécution : exécution du plan d'exécution
    - a. Si les blocs de données sont dans le DBC, OK
    - b. Sinon, il les récupère des fichiers de données
  - III. Phase de fetch : renvoi du résultat de la requête par le processus serveur, au processus utilisateur.

Fonctionnement d'Oracle – Requête UPDATE

- Le serveur BD reçoit une requête UPDATE
  I. Phase de parse : la même que pour une requête SELECT.
  II. Phase d'exécution :
    a. Chargement des blocs à modifier dans le DBC, si elles n'y existent pas
    b. Verrouillage des lignes à modifier
    c. Un Redo Entry est généré pour la requête et est écrit dans le RLB
    d. Modification des blocs dans le DBC, et écriture différée (par le DBW) des modifications sur les fichiers de données
  III. Phase de fetch : pas de résultat renvoyé. Juste un message décrivant le déroulement de l'opération.

Fonctionnement d'Oracle – Commande COMMIT

- Le serveur BD reçoit la commande COMMIT

I. Affectation d'un numéro SCN, par le LGWR, à la transaction validée.

II. Écriture des Redo Entry de la transaction sur les fichiers journaux, par le LGWR.

III. Suppression des informations d'annulation, en relation avec la transaction validée, des segments d'annulation

Fonctionnement d'Oracle – Commande ROLLBACK

- Le serveur BD reçoit la commande ROLLBACK

I. Deux scénarii possibles :

a. Les blocs modifiés (dirty) sont déjà enregistrés dans les fichiers de données.

→ Récupération de l'ancienne image à partir des segments d'annulation

a. Les blocs modifiés sont encore dans le DBC.

→ L'annulation n'engendre aucune écriture sur le disque.

Démarrage de l'instance (1)

- Le démarrage de la base de données passe par trois étapes :

I. Démarrage de l'instance

- Lecture du fichier de ***paramètres***
- Allocation de la ***SGA*** et des ***processus*** selon les valeurs des paramètres

→ État `NOMOUNT` : seuls les ***vues dynamiques du DD relatives à l'instance*** sont consultables (`V$INSTANCE, V$PARAMETER, V$SGA` etc.)

→ Utilisé généralement pour <u>créer une nouvelle BD</u>

→ **`STARTUP NOMOUNT`**

Démarrage de l'instance (2)

II. Démarrage de la base de données

- L'administrateur précise une BD à monter
- L'instance ouvre le fichier de ***contrôle*** (dont le chemin est la valeur du paramètre `CONTROL_FILES`) **.**
- Les chemins des fichiers de ***données*** et ***journaux*** sont lus, mais ***pas ouverts***
- État **MOUNT** : seuls `SYSDBA` et `SYSOPER` peuvent interroger la BD
- La vue ***V$DATABASE*** est interrogeable, mais pas les vues statiques, puisque la base n'est pas ouverte
- Exemple de tâches : **restauration, activation du mode** `ARCHIVELOG`**, déplacement des fichiers de données et de journaux**

→ `STARTUP MOUNT [nom_BD]`

Démarrage de l'instance (3)

III. Ouverture de la base de données

- Ouverture des fichiers de ***données et de journaux***. État `OPEN` : tous les utilisateurs peuvent accéder à la BD. Toutes les **vues** du DD sont accessibles
- Si un tablespace a été mis en `OFFLINE` lors de l'arrêt, il le sera aussi à l'ouverture de la BD
- Si le dernier arrêt a été anormal, alors une opération de récupération est effectuée par `SMON`

→ `STARTUP OPEN [nom_BD]`

- Il est possible d'ouvrir la BD en mode `READ ONLY`

→ `ALTER DATABASE OPEN READ ONLY`

- Pour remettre la BD en mode `READ WRITE`

→ `ALTER DATABASE OPEN READ WRITE`

Démarrage de l'instance (4)

- Démarrage en mode force
  - Si le démarrage normal ne se produit pas, on peut forcer le démarrage
  - `STARTUP FORCE`
- Démarrage en mode restreint
  - Donne l'accès uniquement aux utilisateurs ayant le privilège **RESTRICTED SESSION**
  - Option utilisée si l'administrateur a des tâches de maintenance à exécuter
    - → `STARTUP RESTRICT`
    - ❖ Pour désactiver le mode restreint
      - → ALTER SYSTEM DISABLE RESTRICTED SESSION

Démarrage de l'instance (5)

- Démarrage suivant un PFILE
  - Oracle démarre à partir de `spfile<sid>.ora`, si non trouvé, `spfile.ora`, sinon `init<sid>.ora`
  - Pour démarrer à partir d'un fichier PFILE spécifié par l'administrateur

    → `STARTUP PFILE=nom_pfile`

- Syntaxe complète de la commande STARTUP

→ `STARTUP [NOMOUNT|MOUNT [nom_bd]|OPEN [nom_bd]] [FORCE] [RESTRICT] [PFILE=nom_pfile]`

Arrêt de la base de données (1)

- **La fermeture de la base passe par les mêmes étapes de son ouverture (mount, nomount et arrêt), *mais ne s'arrête pas sur un de ces états***
- Il existe 4 types d'arrêt:

I. Arrêt normal

- Oracle ne permet ***plus*** l'ouverture des ***nouvelles sessions***
- Oracle ***attend*** que le ***dernier*** utilisateur soit ***déconnecté*** pour arrêter l'instance
- Il s'agit d'un arrêt propre (***checkpoint*** avant fermeture)

→ **`SHUTDOWN NORMAL`**

Arrêt de la base de données (2)

- Il existe 4 types d'arrêt:

I. Arrêt normal

II. Arrêt Transactionnel

- Oracle ne permet ***plus*** le lancement de ***nouvelles transactions***
- Oracle ***attend*** que la ***validation/annulation*** des transactions courantes
- Il s'agit d'un arrêt propre (***checkpoint*** avant fermeture)

→ **`SHUTDOWN TRANSACTIONAL`**

Arrêt de la base de données (3)

- Il existe 4 types d'arrêt:

I. Arrêt normal

II. Arrêt transactionnel

III. Arrêt immédiat

- Oracle ***annule*** toutes les ***transactions courantes*** et ferme la BD
- Il s'agit d'un arrêt propre (***checkpoint*** avant fermeture)

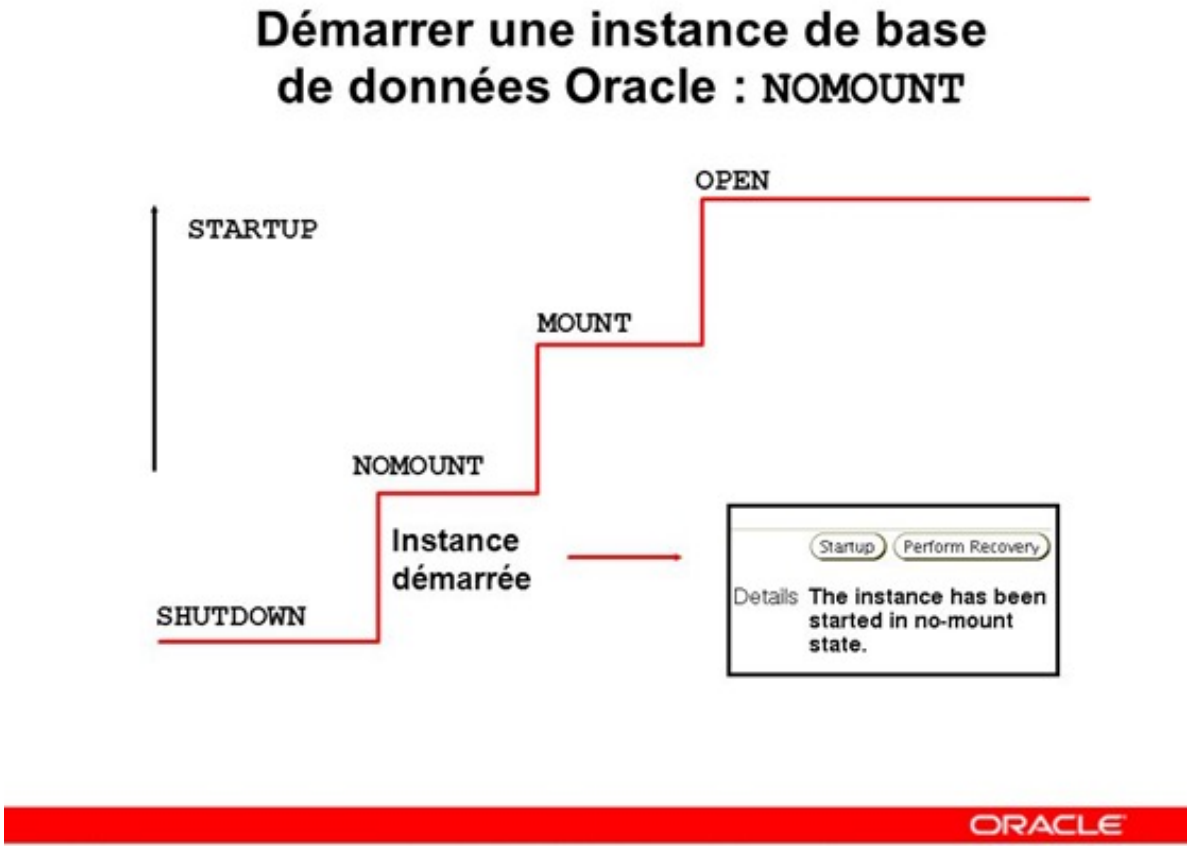
→ **SHUTDOWN IMMEDIATE**

Arrêt de la base de données (4)

- Il existe 4 types d'arrêt:
  I. Arrêt normal
  II. Arrêt transactionnel
  III. Arrêt immédiat
  IV. Arrêt d'abandon
    - Oracle ferme la BD sans faire de checkpoint. C'est l'arrêt le plus ***brutal***.
    - Au prochain démarrage, une récupération des données, par le SMON, est effectuée

    → **SHUTDOWN ABORT**

Etapes de démarrage de la BDD



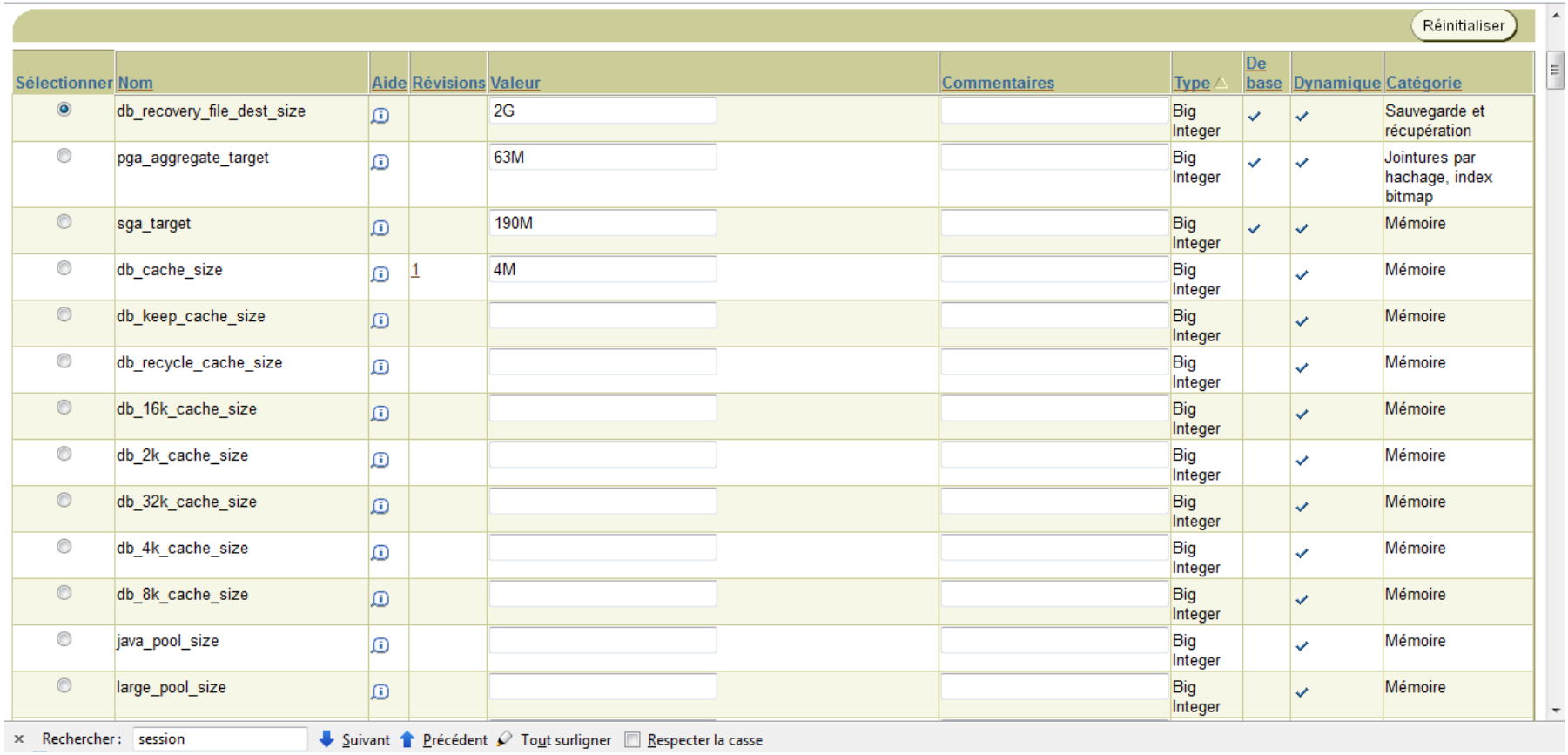
Gestion de l'instance

- Gérer l'instance revient principalement à la paramétrer
- Il existe deux types de paramètres:
  1. **Dynamique** : sa modification est **prise en compte** en cours de ***fonctionnement de l'instance***
  2. **Statique** : sa ***modification*** est prise en compte ***après redémarrage***
- Un **paramètre**, qu'il soit statique ou dynamique est modifié:
  1. Via SQL (isqlplus, sqlplus)

     ```
     ALTER SYSTEM SET param=val;
     ```

  1. Via Enterprise Manager

Gestion de l'instance



Réinitialiser

Sélectionner	Nom	Aide	Révisions	Valeur	Commentaires	Type	De base	Dynamique	Catégorie
◉	db_recovery_file_dest_size			2G		Big Integer	✓	✓	Sauvegarde et récupération
○	pga_aggregate_target			63M		Big Integer	✓	✓	Jointures par hachage, index bitmap
○	sga_target			190M		Big Integer	✓	✓	Mémoire
○	db_cache_size		1	4M		Big Integer		✓	Mémoire
○	db_keep_cache_size					Big Integer		✓	Mémoire
○	db_recycle_cache_size					Big Integer		✓	Mémoire
○	db_16k_cache_size					Big Integer		✓	Mémoire
○	db_2k_cache_size					Big Integer		✓	Mémoire
○	db_32k_cache_size					Big Integer		✓	Mémoire
○	db_4k_cache_size					Big Integer		✓	Mémoire
○	db_8k_cache_size					Big Integer		✓	Mémoire
○	java_pool_size					Big Integer		✓	Mémoire
○	large_pool_size					Big Integer		✓	Mémoire

× Rechercher : session Suivant Précédent Tout surligner Respecter la casse

Gestion de l'instance

- L'ensemble des paramètres sont consultables sur `V$PARAMETER`
- L'attribut `ISSYS_MODIFIABLE` peut avoir la valeur :
  1. ***FALSE*** : le paramètre est **statique**, sa modification s'applique après **redémarrage**
  2. ***IMMEDIATE*** : le paramètre est **dynamique**, et **modifié** pour la **session courante**
  3. **DEFERRED** : le paramètre est **dynamique**, et modifié pour la **session future**
- Syntaxe de l'instanciation des paramètres

```
ALTER SYSTEM SET param=val [COMMENT='txt'] [DEFERRED]
[SCOPE=MEMORY|SPFILE|BOTH]
```

- L'attribut **`ISSES_MODIFIABLE`** peut avoir la valeur :
  1. ***TRUE*** : le paramètre est modifiable au niveau de la session
  2. ***FALSE*** : le paramètre est **non modifiable au niveau de la session**
     `ALTER SYSTEM SET param=val`

Gestion de l'instance

Afficher la valeur actuelle, et celle sur le fichier SPFILE du paramètre SGA_TARGET ?

```
SELECT p.name, p.display_value,
sp.display_value
FROM v$parameter p, v$spparameter sp
WHERE p.name=sp.name AND
p.name='sga_target';
```

- Pour modifier un **paramètre statique** sur un **SPFILE**

1. Si le serveur en arrêt:

   → Démarrer en `NOMOUNT`, `ALTER SYSTEM` ensuite, **redémarrage**

2. Si le serveur fonctionne

   → `ALTER SYSTEM` avec un `SCOPE=SPFILE` ensuite, redémarrage

`V$PARAMETER2`, `V$SYSTEM_PARAMETER2` et `V$SPPARAMETER2` affichent sur plusieurs lignes les paramètres qui ont une liste de valeurs.

- Différence entre les vues des paramètres

→ **V$PARAMETER** : valeurs actuelles des paramètres dans la session courante.

→ **V$SYSTEM_PARAMETER** : valeurs actuelles des paramètres dans l'instance (toutes sessions confondues)

→ **V$SPPARAMETER** : contenu du fichier SPFILE avec lequel le serveur a démarré. Contenu vide si le serveur a démarré avec un PFILE

L’environnement oracle mutualisé(multitenant)

- L’architecture mutualisée permet à une BDD oracle de fonctionner comme un conteneur BD mutualisé (CDB) qui peut avoir 0, 1 ou plusieurs BDD PDBs (pluggable)
- Une PDB est une collection portable de schémas, et des objets de schéma.
- Donc grâce à cette option, une instance Oracle peut héberger plusieurs bases.

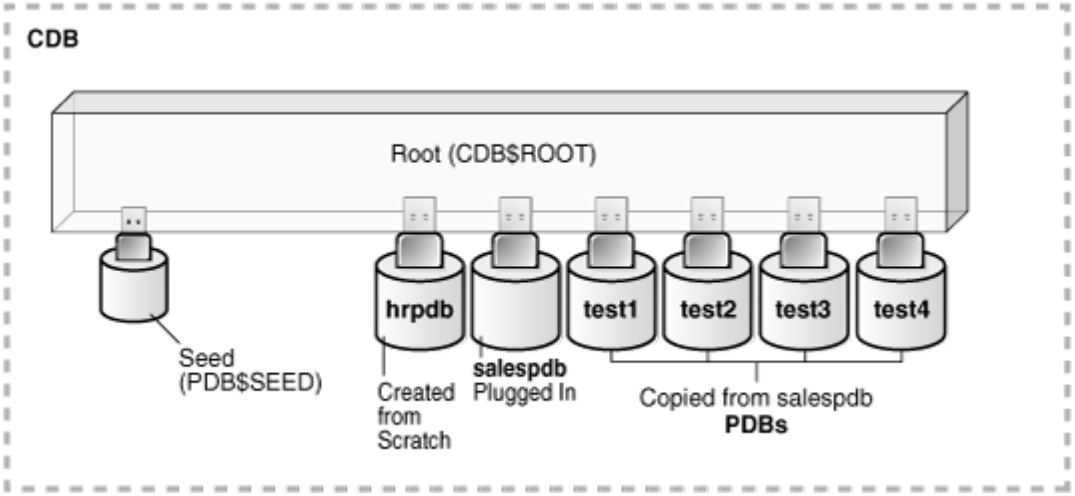
Les composants d'un CDB

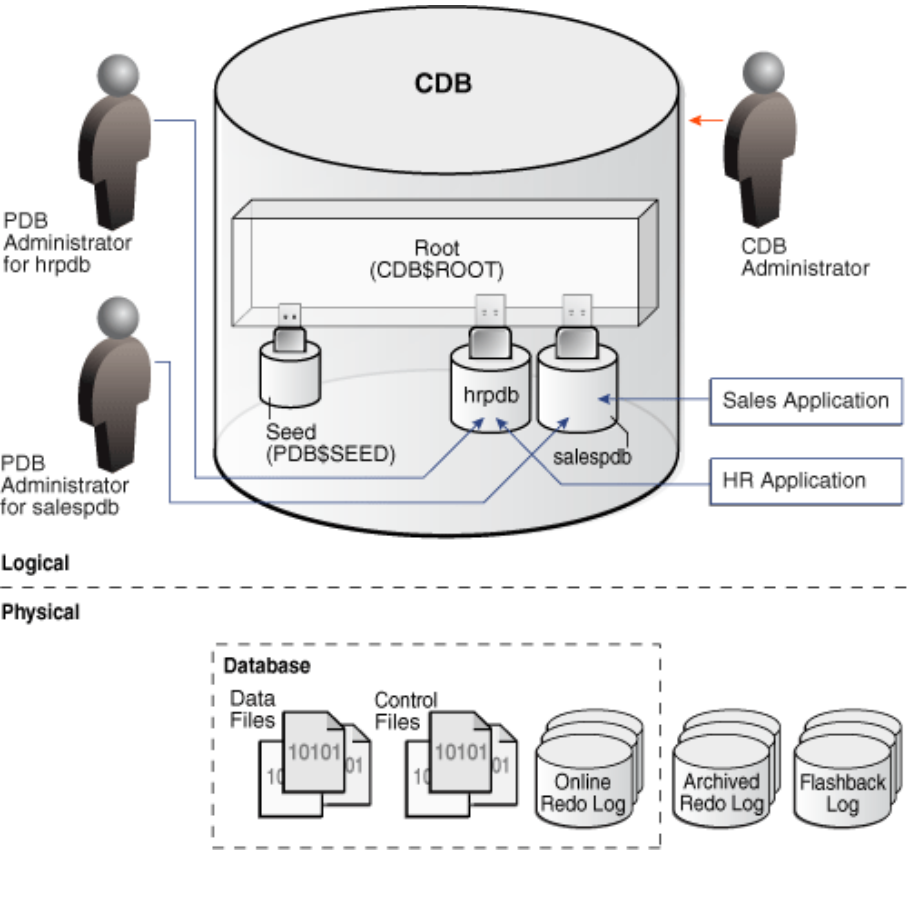
- ROOT:
  - Root nommé CDB$ROOT stocke les métadonnées et les utilisateurs communs fournis par Oracle.
  - Un CDB a exactement un seul Root
- SEED
  - Nommé PDB$SEED est un modèle que vous pouvez utiliser pour créer une nouvelle PDB.
  - Un CDB a exactement un seul seed
- PDBs
  - Une PDB apparait aux utilisateurs et aux applications comme si elle était une non-CDB

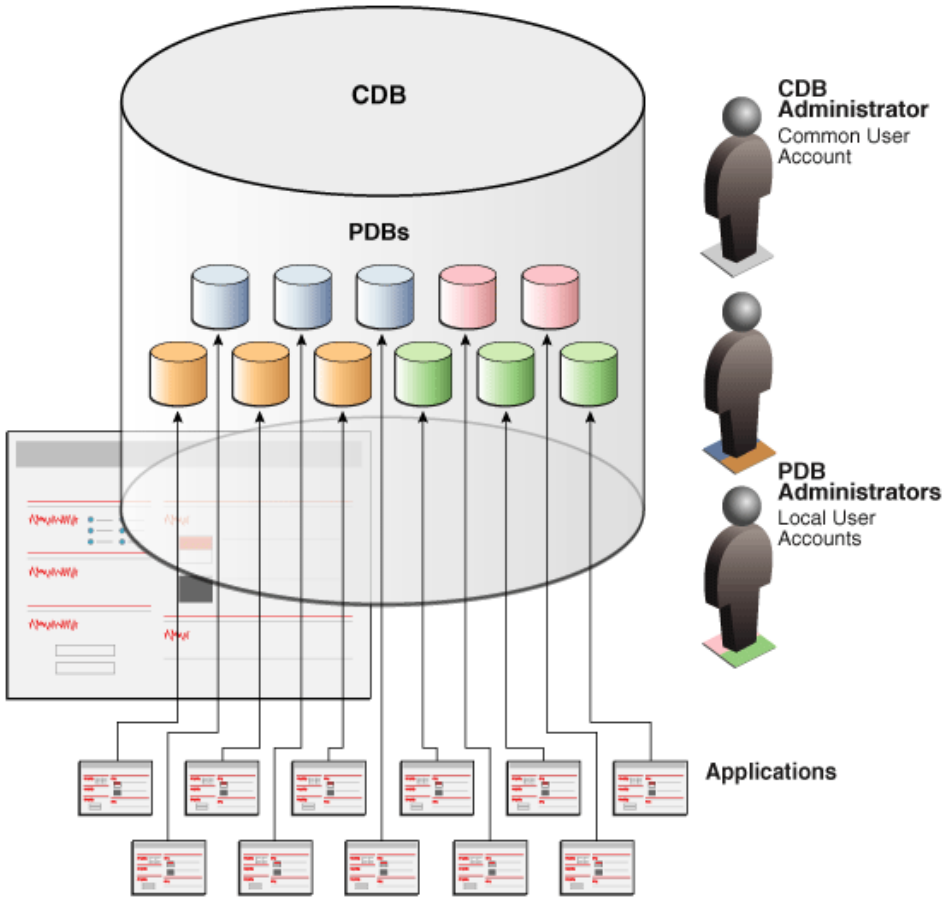
Les composants d'un CDB

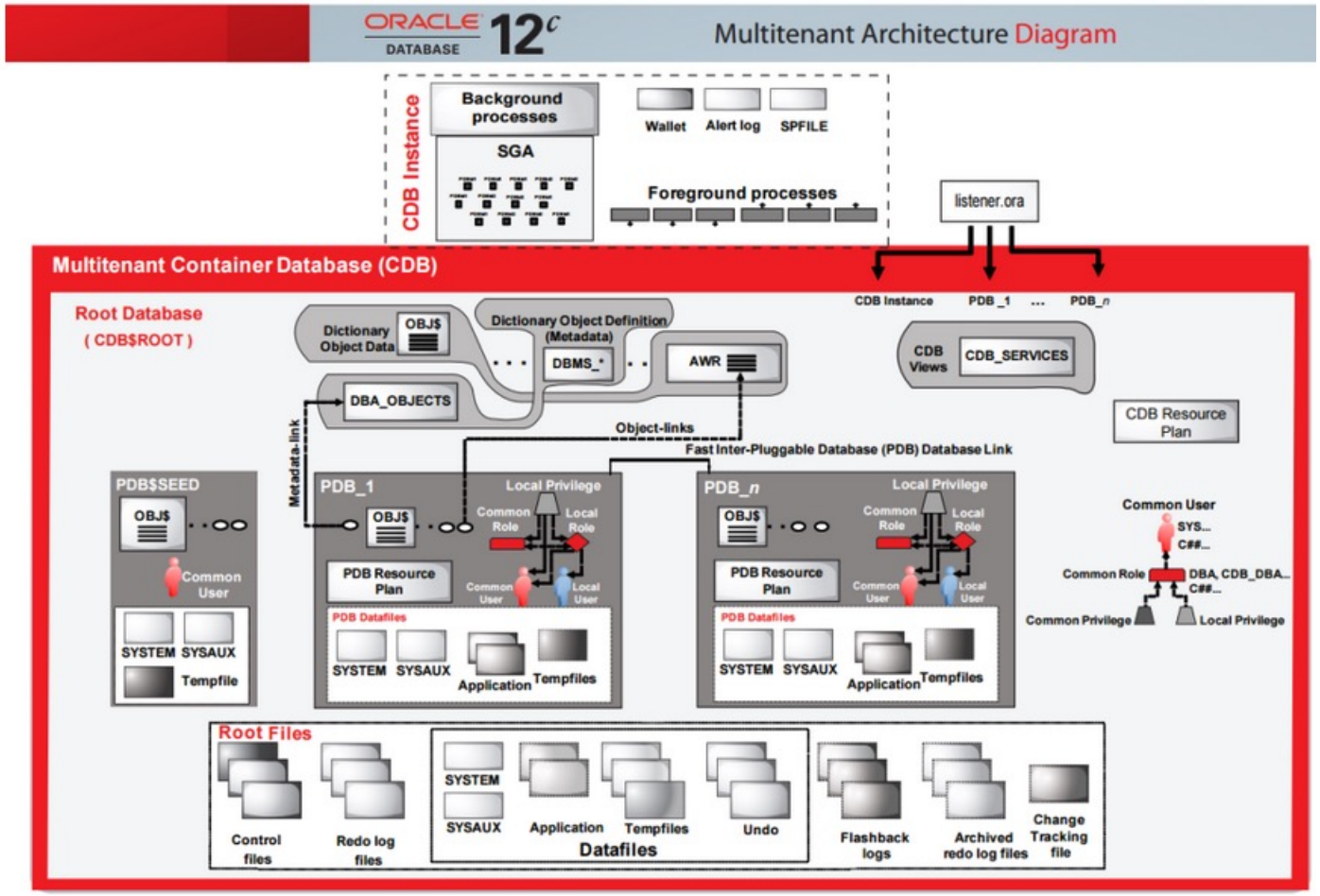
Chacun de ces composants est appelé conteneur(container).

Chaque conteneur possède un **ID** unique et un **nom dans un CDB**.




CDB
PDB Administrator for hrpdb
CDB Administrator
Root (CDB$ROOT)
hrpdb
Seed (PDB$SEED)
salespdb
Sales Application
HR Application
PDB Administrator for salespdb
Logical
Physical
Database
Data Files
Control Files
Online Redo Log
Archived Redo Log
Flashback Log

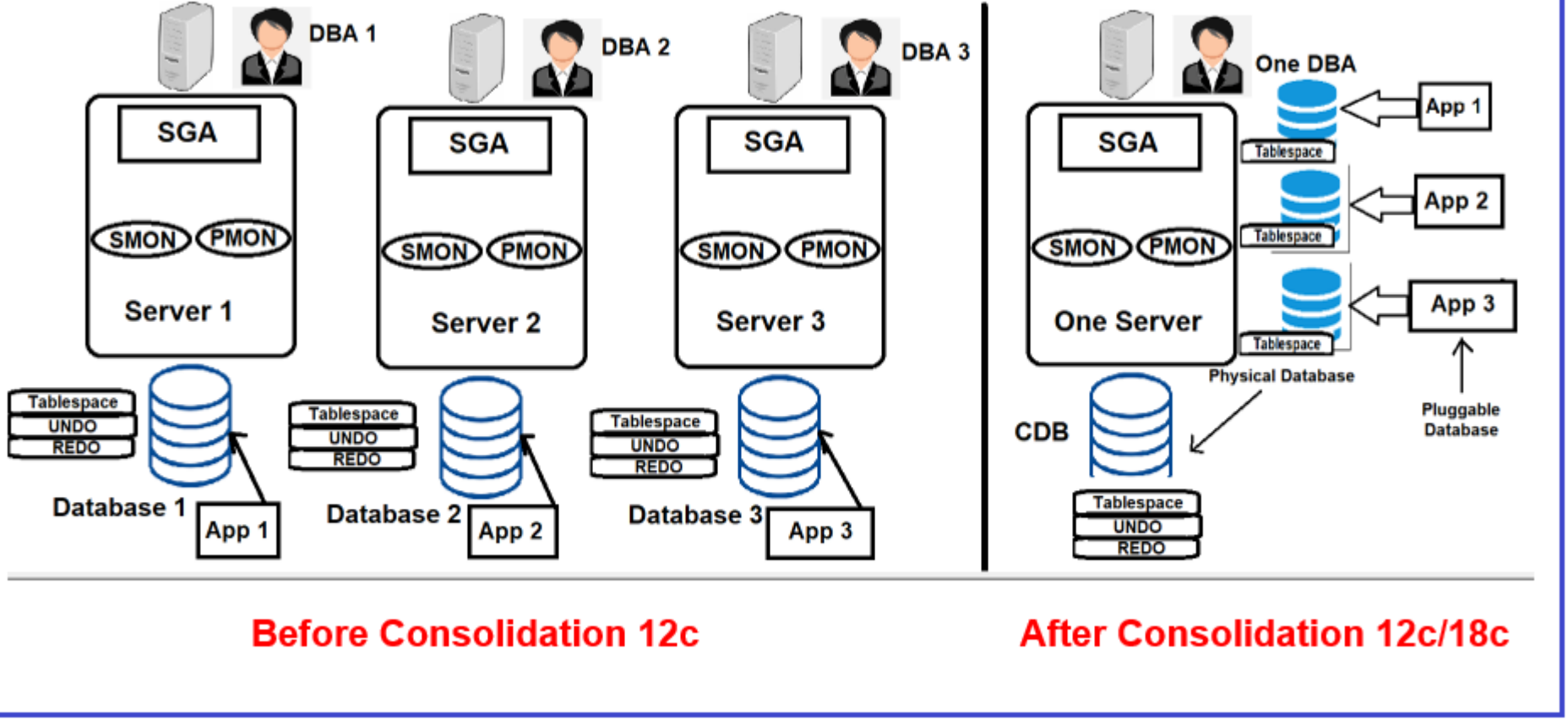

CDB
PDBs
CDB Administrator
Common User Account
PDB Administrators
Local User Accounts
Applications


ORACLE DATABASE 12c
Multitenant Architecture Diagram
CDB Instance
Background processes
SGA
Wallet
Alert log
SPFILE
Foreground processes
listener.ora
Multitenant Container Database (CDB)
CDB Instance
PDB_1
...
PDB_n
Root Database
(CDB$ROOT)
Dictionary Object Data
OBJ$
Dictionary Object Definition (Metadata)
DBMS_*
AWR
DBA_OBJECTS
CDB Views
CDB_SERVICES
CDB Resource Plan
Object-links
Metadata-link
Fast Inter-Pluggable Database (PDB) Database Link
PDB$SEED
OBJ$
Common User
SYSTEM
SYSAUX
Tempfile
PDB_1
Local Privilege
Common Role
Local Role
PDB Resource Plan
Common User
Local User
PDB Datafiles
SYSTEM
SYSAUX
Application
Tempfiles
PDB_n
Local Privilege
Common Role
Local Role
PDB Resource Plan
Common User
Local User
PDB Datafiles
SYSTEM
SYSAUX
Application
Tempfiles
Common User
SYS...
C##...
Common Role
DBA, CDB_DBA...
C##...
Common Privilege
Local Privilege
Root Files
Control files
Redo log files
SYSTEM
SYSAUX
Application
Tempfiles
Undo
Datafiles
Flashback logs
Archived redo log files
Change Tracking file

L’environnement oracle mutualisé(multitenant)

- Les avantages mis en avant sont :
  - Il y a une entière compatibilité avec les bases non-CDB : aucune modification de code n'est nécessaire
  - Meilleure utilisation des ressources de la machine (mutualisation de la mémoire partagée et des metadata oracle)
  - Upgrade plus facile car on upgrade la CDB. Autre methode : on plug la PDB sur une CDB déjà upgradée
  - Support d'Oracle RAC sur une CDB
  - Support de DataGuard sur une CDB
  - Support d'EM sur une CDB
  - Support du Resource Manager sur une CDB
  - Séparation des metadatas utilisateur des metadatas Oracle (tablespaces SYSTEM et SYSAUX séparés)
  - Remarque : l'accès aux metadata des objets Oracle depuis une PDB se fait par des pointeurs internes vers les metadatas de la CDB.

Quel est l'intérêt ?



Quel est l'intérêt ?

- Moins de ressources (1 seule instance)
  - Moins de RAM
  - Moins de CPU (car moins de processus)
  - Moins de STOCKAGE (1 seule fois le dictionnaire interne)
- Plus de souplesse
  - Duplication d'une base de données « pluggable » en 1 ligne de commande sous « sqlplus »
  - On peut « attacher » ou « détacher » une « Pluggable Database » d'un « Container » à un autre indépendamment de la version Oracle du « Container » (c'est d'ailleurs une technique de migration supportée)
  - On peut donner des droits d'administration uniquement des « Pluggable Database » aux administrateurs d'application correspondants (nouveau rôle)
  - Les patchs ne s'appliquent qu'aux « containers »
- Réduction des couts
  - Moins de ressources = moins de « hardware » et moins de ressources humaines(DBAs)
  - Moins de « hardware » = moins de licences
  - Délégation possible d'une partie de l'administration des bases de données (aux responsables applicatifs)

Oracle metadata vs User data

A multitenant container database has three types of containers:

The Oracle supplied container is called the root container (**CDB$ROOT**) and consists of just **Oracle metadata** (and maybe a **little bit of user data**) and **common users**. Each CDB has one root.

The seed container is named **PDB$SEED**, and there is one of these per CDB. The purpose of the seed container isn't to store user data—it's there for you as a **template** to **create PDBs**.

The **user container** , which is actually called a **pluggable database** (or **PDB**), consists of **user metadata** and **user data**.

Each of these—the **root**, the **seed**, and the **PDB(s**)—is called a **container**, and each container has a unique container ID (**CON_ID**) and container name (**CON_NAME**). **Each PDB** also has a **globally unique identifier** (**GUID**)

Oracle metadata vs User data

- The idea behind the concept of a container is to **separate Oracle metadata** and **user data** by placing the two types of data **into separate containers**. That is, the system and user data are separated.
- There's a **SYSTEM tablespace** in **both** the **central container** and the **PDB containers**, however, **they contain different types of data**. The ***root container consists of Oracle metadata*** whereas the **PDB container's SYSTEM tablespace contains just user metadata**.
- ***The Oracle metadata isn't duplicated by storing it in each PDB***—it's stored in a central location ***for use by all the PDBs that are part of that CDB***. **The CDBs contain pointers to the Oracle metadata in the root container**, **thus allowing the PDBs to access these system objects without duplicating them in the PDBs.**
- A CDB has similar background processes and files as a normal non-CDB database. However, some of the processes and files are common for both a CDB and its member PDB databases, and some aren't

Common Entities between CDB and PDBs

Background processes There's a single set of background processes for the CDB. The PDBs don't have any background processes attached to them.

Redo log files These are common for the entire CDB, with Oracle annotating the redo data with the identity of the specific PDB associated with the change. There's one active online redo log for a single-instance CDB or one active online redo log for each instance of an Oracle RAC CDB. A CDB also has a single set of archived redo log files.

Memory You allocate memory only to the CDB, because that's the only instance you need in a multitenant database.

Control files These are common for the entire CDB and will contain information that reflects the changes in each PDB.

Oracle metadata All Oracle-supplied packages and related objects are shared.

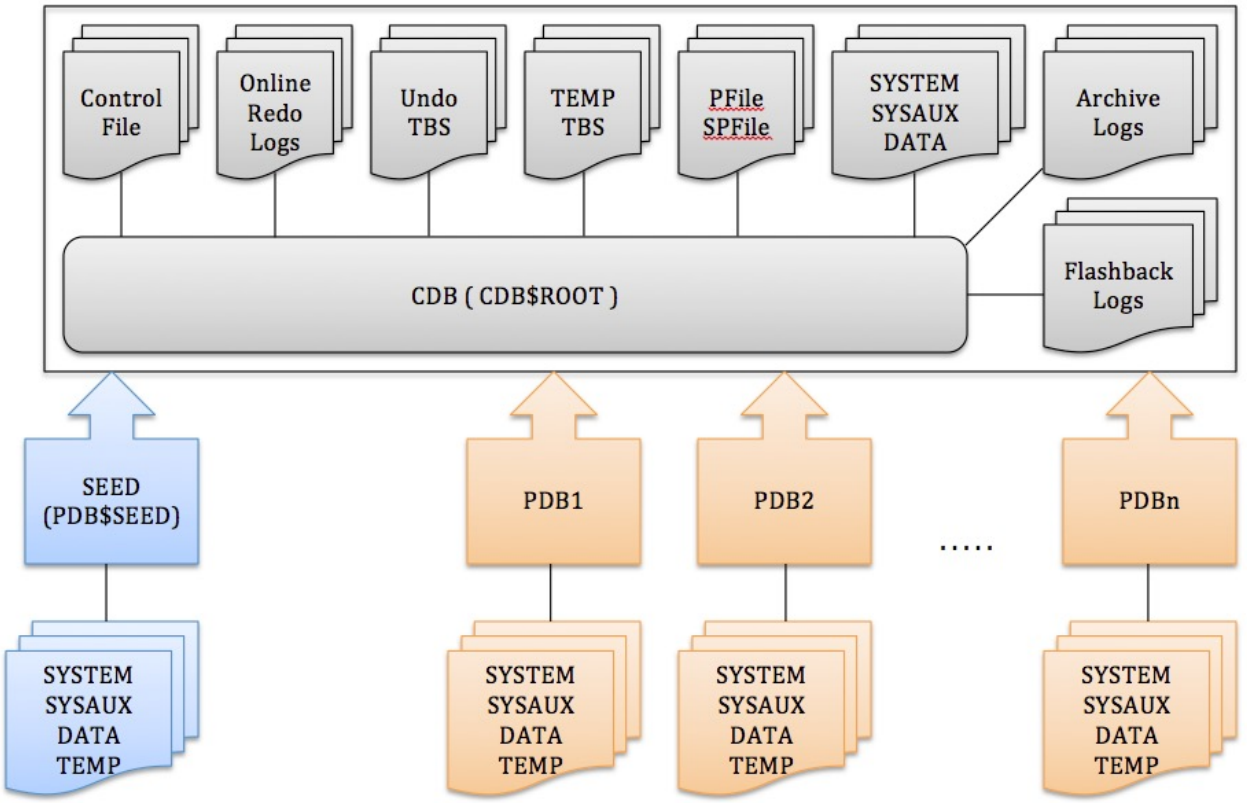
Temporary tablespace There's a common temporary tablespace for an entire CDB. Both the root and all the PDBs can use this temporary tablespace. This common tablespace acts as the default TEMP tablespace. In addition, each PDB can also have a separate temporary tablespace for its local users.

Undo tablespace All PDBs use the same undo tablespace. There's one active undo tablespace for a single-instance CDB or one active undo tablespace for each instance of an Oracle RAC CDB.

Entities Exclusive for PDBs

- **Tablespaces for the applications tables and indexes** These application tablespaces that you'll create are specific to a PDB and aren't shared with other PDBs, or the central CDB. **The data files that are part of these tablespaces constitute the primary physical difference between a CDB and a non-CDB**. Each data file is associated to a specific container.
- **Local temporary tablespaces** Although the temporary tablespace for the CDB is common to all containers, each PDB can also create and use its own temporary tablespaces for its local users.
- **Local users and local roles** Local users can connect only to the PDB where the users were created. A common user can connect to all the PDBs that are part of a CDB.
- **Local metadata** The local metadata is specific to each application running in a PDB and therefore isn't shared with other PDBs.
- **PDB Resource Manager Plan** These plans allow resource management within a specific PDB. There is separate resource management at the CDB level.

CDB vs PDBs



Above diagram depicts what a CDB contains and what a PDB contains.

SYSAUX TABLESPACE

The **SYSAUX** tablespace stores data for auxiliary applications such as the Log Miner, Workspace Manager, Oracle Data Mining, Oracle Streams, and many other Oracle tools.

This tablespace is automatically created if you use the **DataBase Creation Assistant(DBCA)** software to build an Oracle database.

Like the **SYSTEM** tablespace, **SYSAUX** requires a higher level of security, and it cannot be dropped or renamed.

Do not allow user objects to be stored in **SYSAUX**. This tablespace should only store system specific objects.

This is a permanent tablespace.